



Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

LEARNER GUIDE

For

Developing AI Apps and Agents on Azure (AI-103)

TGS Ref No: TGS-2023036651

Conducted by

Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

Version 5.0

DOCUMENT VERSION CONTROL RECORD

Version Number	Effective Date of Release	Summary of Included Changes	Author
2.0	26 Aug 2026	Prior AI-102 courseware release	Tertiary Infotech Academy Pte Ltd
3.0	27 Sep 2026	AI-103 title transition	Tertiary Infotech Academy Pte Ltd
4.0	27 Sep 2026	Five-domain AI-103 courseware and original Tertiary labs	Tertiary Infotech Academy Pte Ltd
5.0	27 Sep 2026	Pinned Microsoft Learning exercises and assets copied unchanged; ten selected labs aligned to AI-103 domains, slides, and WSQ evidence.	Tertiary Infotech Academy Pte Ltd

TABLE OF CONTENTS

01	How to Use This Guide
02	Before You Start
03	Lab 01 - Plan a Foundry project
04	Lab 02 - Evaluate models and release guardrails
05	Lab 03 - Build a generative chat app
06	Lab 04 - Build a Foundry agent
07	Lab 05 - Use a custom tool in an agent
08	Lab 06 - Develop a vision-enabled chat app
09	Lab 07 - Analyze text
10	Lab 08 - Develop a text analysis agent
11	Lab 09 - Extract multimodal information
12	Lab 10 - Build a knowledge mining solution
13	Quick Command Reference
14	Assessment Flow and Support

How to Use This Guide

Use the trainer deck to understand mechanisms and decisions. Use this guide for the detailed click paths, commands, code, expected results, diagnostics, and evidence required in each lab.

Authority	Current reference
Approved course	Developing AI Apps and Agents on Azure (AI-103) · TGS-2023036651 · 2 days / 16 hours
Course enquiries	https://www.tertiarycourses.com.sg/
AI-103 study guide	Skills measured from 16 Apr 2026
Microsoft course	AI-103T00-A: Develop AI apps and agents on Azure
LMS	https://lms-tms.tertiaryinfotech.com/

Before You Start

- Use the Azure subscription or lab environment supplied by the trainer.
- Install current Azure CLI/Python tooling only when the lab requires it.
- Authenticate with Microsoft Entra ID where supported; never save live keys in the repository.
- Use synthetic data and remove training resources after evidence has been captured.
- For every lab, retain request IDs, deployment/model versions, screenshots or JSON outputs, and a short interpretation.

Lab 01 - Plan a Foundry project

Source folder: labs/lab-01-plan-a-foundry-project

Objectives

- Complete the exact Microsoft Learning exercise for the Plan and manage domain.
- Capture a working result and explain its product-development implications.

Source

The complete, unchanged Microsoft exercise and assets are in [official/mslearn-ai-studio/Instructions/Exercises/01-Explore-ai-studio.md](../official/mslearn-ai-studio/Instructions/Exercises/01-Explore-ai-studio.md). The source revision and SHA-256 hashes are recorded in [OFFICIAL-SOURCE-MANIFEST.json](../OFFICIAL-SOURCE-MANIFEST.json). Read the source file for every numbered instruction; this course wrapper does not alter those steps.

Steps

Prepare

Read the source exercise prerequisites and use a trainer-provided Azure subscription or approved lab environment. Do not commit your own .env values.

Complete the official exercise

Follow the numbered steps in [01-Explore-ai-studio.md](../official/mslearn-ai-studio/Instructions/Exercises/01-Explore-ai-studio.md) exactly. Use its companion Labfiles or labfiles directory in the same source snapshot.

Capture evidence

Record the deployment or resource names, model and API versions, input, output, latency or quality measure, and one failure or limitation. Remove personal data and live keys.

Validation

The official exercise completes with its expected output; a second learner can reproduce the result from the source instructions and the recorded environment.

Timing

Approximately 30 minutes for the official exercise; trainer discussion and WSQ evidence review are scheduled separately.

Unchanged Microsoft Learning exercise instructions

Exact Markdown source and companion assets:

labs/official/mslearn-ai-studio/Instructions/Exercises/01-Explore-ai-studio.md. Follow the numbered instructions below with the source assets. Microsoft Learning MIT license and pinned revision are in labs/official/ and the source manifest.

In this exercise, you use Microsoft Foundry portal to create a project, ready to build an AI solution.

This exercise takes approximately 30 minutes.

> Note: Some of the technologies used in this exercise are in preview or in active development. You may experience some unexpected behavior, warnings, or errors.

Prerequisites

Before starting this exercise, ensure you have:

- An active [Azure subscription](https://azure.microsoft.com/pricing/purchase-options/azure-account)
- [Visual Studio Code](https://code.visualstudio.com/) installed
- [Python version **3.13.xx**](https://www.python.org/downloads/release/python-31312/) installed*
- [Git](https://git-scm.com/install/) installed and configured
- [Azure CLI](https://learn.microsoft.com/cli/azure/install-azure-cli?view=azure-cli-latest) installed

> \ Python 3.14 is available, but some dependencies are not yet compiled for that release. The lab has been successfully tested with Python 3.13.12.

Create a Microsoft Foundry project

Microsoft Foundry uses projects to organize models, resources, data, and other assets used to develop an AI solution.

1. In a web browser, open the [Microsoft Foundry portal](https://ai.azure.com) at `https://ai.azure.com` to start building; signing in using your Azure credentials. Close any tips or quick start panes that are opened the first time you sign in.
2. If it is not already enabled, in the tool bar at the top of the page, enable the **New Foundry** option. Then, create a new project with a unique name; expanding the **Advanced options** area to specify the following settings for your project:
 - Foundry resource: Use the default name for your resource (usually {project_name}-resource)
 - Subscription: Your Azure subscription
 - Resource group: Create or select a resource group

- Region: Select any of the AI Foundry recommended regions in [this list](https://learn.microsoft.com/azure/foundry/openai/how-to/responses#region-availability){:target="_blank"}

> Tip: Make a note of the region you selected. You'll need it later!

3. Select **Create**. Wait for your project to be created.

When it is ready, the project home page will open.

![Screenshot of the Foundry project home page.](../media/foundry-portal-home.png)

Deploy and test a model

At the core of any generative AI project, there's at least one generative AI model.

4. Now you're ready to explore models. On the **Discover** page, select the **Models** tab to view the Microsoft Foundry model catalog.

5. Search for the `gpt-5.2` model, and then select it in the search results to view its model card.

Model cards provide information about models to help you understand their capabilities and limitations, and determine if they are suitable for your requirements.

![Screenshot of the gpt-5.2 model card.](../media/gpt5.2-details.png)

6. Select **Deploy** with the default settings to create a deployment of the model.

Model deployments enable you to work with a model in your project.

When the model has been deployed, the model playground will open automatically so you can test your model:

![Screenshot of the Foundry project model playground.](../media/ai-foundry-model-playground.png)

7. In the **Instructions** box, enter the following instructions:

text

You are an AI assistant that can provide information and advice about AI software development.

8. In the chat window, enter a query such as `Describe three key considerations for working with Large Language Models for AI application development.` and view the response:

Hopefully the model provided some key considerations for you to think about!

View Foundry Azure resource and project endpoints

9. In the Foundry portal, in the top menu bar, select **Manage**.

The management center is where you can view and administer your projects and their parent resources.

![Screenshot of the Manage tab in Foundry portal.](../media/ai-foundry-manage.png)

- The resource level relates to the Foundry resource that was created in Azure to support your project. This resource includes connections to Foundry Services and models; and provides a central place to manage user access to AI development projects.

- The project level relates to your individual project, where you can add and manage project-specific resources. A resource can support multiple projects (the first one created is the resource's default project).

10. Select the link to the **Parent resource** associated with the project.

The resource configuration details should be displayed.

Note that the Foundry resource has an endpoint, through which client applications can access resource-level functionality (such as Foundry Tools that are shared across all projects in the resource).

11. In the top menu bar, select **Home** to return to the project home page.

12. Note the key, project endpoint, and Azure OpenAI endpoint.

This information is used to connect to your project-level resources from client applications.

- The key is used for key-based authentication to models and tools (though in most production scenarios you should consider using Microsoft Entra ID authentication based on authenticated user and application identities).

- The project endpoint is used to access models provided directly in Foundry (including OpenAI models) using the OpenAI Responses API, and to access Foundry-specific APIs (such as the Foundry Agent service).

- The OpenAI endpoint is used to access models using OpenAI APIs, including the Chat Completions API and the Responses API.

Install the Foundry Toolkit extension for Visual Studio Code

As a developer, you may spend some time working in the Foundry portal; but you're also likely to spend a lot of time in Visual Studio Code. The Foundry Toolkit extension provides a convenient way to work with Foundry project resources without leaving the development environment.

13. Start Visual Studio Code

14. In the navigation bar on the left, view the **Extensions** page.

15. Search the extensions marketplace for 'Foundry Toolkit', and install the **Foundry Toolkit for VS Code** extension.

The extension may take a minute or so to install.

16. After installing the extension, select the **Foundry Toolkit** page in the left navigation bar; and wait for it to load.

![Screenshot of the Foundry Toolkit Visual Studio Code extension.](../media/foundry-vs-extension.png)

17. In the Foundry Toolkit pane, expand **Microsoft Foundry Resources** and set the default project by connecting to Azure (signing in with your credentials) and selecting the Foundry project you created previously.

18. After setting the default project, expand the project, expand **Models**, and select the **gpt-5.2** model you deployed previously.

You can view the model deployment details here.

![Screenshot of a model in the Foundry Toolkit Visual Studio Code extension.](../media/vscode-extension-model.png)

19. In the Foundry Toolkit pane, in the **Developer Tools** section, expand **Build** and select **Model playground**. Then select the **gpt-5.2** model (if it is not already selected).

An interactive playground in which you can test the model is opened in Visual Studio Code.

![Screenshot of the model playground in Visual Studio Code.](../media/vscode-model-playground.png)

Summary

In this exercise, you've created a Microsoft Foundry and explored it in the Foundry portal. You've also explored the Foundry Toolkit extension in Visual Studio Code, which provides a convenient way for developers to work with Foundry projects and their assets.

Clean up

If you've finished exploring Foundry portal, you should delete the resources you have created in this exercise to avoid incurring unnecessary Azure costs.

20. In the [Azure portal](https://portal.azure.com) at `https://portal.azure.com`, view the contents of the resource group where you deployed the resources used in this exercise.

21. On the toolbar, select **Delete resource group**.

22. Enter the resource group name and confirm that you want to delete it.

Acceptance Evidence

Complete the pinned Microsoft Learning exercise [official/mslearn-ai-studio/Instructions/Exercises/01-Explore-ai-studio.md](https://mslearn-ai-studio.azure.com/Instructions/Exercises/01-Explore-ai-studio.md); capture its expected output, relevant version/endpoint, one measured result, and an explained failure or limitation.

Troubleshooting Checklist

23. Capture the full error and request/operation ID before retrying.

24. Confirm the endpoint, API/model version, region, identity, and RBAC scope.

25. Validate payload/schema fields and input content type.

26. Check quota, rate limits, network resolution, private connectivity, and service status.

27. Record the corrective action and rerun the acceptance check.

Lab 02 - Evaluate models and release guardrails

Source folder: labs/lab-02-evaluate-models-and-release-guardrails

Objectives

- Complete the exact Microsoft Learning exercise for the Plan and manage domain.
- Capture a working result and explain its product-development implications.

Source

The complete, unchanged Microsoft exercise and assets are in [\[official/mslearn-ai-studio/Instructions/Exercises/02-model-catalog-evaluation.md\]\(../official/mslearn-ai-studio/Instructions/Exercises/02-model-catalog-evaluation.md\)](https://official.mslearn-ai-studio.azure.com/Instructions/Exercises/02-model-catalog-evaluation.md). The source revision and SHA-256 hashes are recorded in [\[OFFICIAL-SOURCE-MANIFEST.json\]\(../OFFICIAL-SOURCE-](https://official.mslearn-ai-studio.azure.com/Instructions/Exercises/02-model-catalog-evaluation.md)

MANIFEST.json). Read the source file for every numbered instruction; this course wrapper does not alter those steps.

Steps

Prepare

Read the source exercise prerequisites and use a trainer-provided Azure subscription or approved lab environment. Do not commit your own .env values.

Complete the official exercise

Follow the numbered steps in [02-model-catalog-evaluation.md](../official/mslearn-ai-studio/Instructions/Exercises/02-model-catalog-evaluation.md) exactly. Use its companion Labfiles or labfiles directory in the same source snapshot.

Capture evidence

Record the deployment or resource names, model and API versions, input, output, latency or quality measure, and one failure or limitation. Remove personal data and live keys.

Validation

The official exercise completes with its expected output; a second learner can reproduce the result from the source instructions and the recorded environment.

Timing

Approximately 45 minutes for the official exercise; trainer discussion and WSQ evidence review are scheduled separately.

Unchanged MicrosoftLearning exercise instructions

Exact Markdown source and companion assets:

labs/official/mslearn-ai-studio/Instructions/Exercises/02-model-catalog-evaluation.md. Follow the numbered instructions below with the source assets. MicrosoftLearning MIT license and pinned revision are in labs/official/ and the source manifest.

The Microsoft Foundry model catalog serves as a central repository where you can explore and use a variety of models, facilitating the creation of your generative AI scenario. In this exercise, you'll explore the model catalog, compare models using benchmarks, test models in the model playground, and run an evaluation using a synthetic dataset.

This exercise will take approximately 45 minutes.

> Note: Some of the technologies used in this exercise are in preview or in active development. You may experience some unexpected behavior, warnings, or errors.

Prerequisites

To complete this exercise, you need:

- An [Azure subscription](https://azure.microsoft.com/free/) with permissions to create AI resources.

Create a Microsoft Foundry project

Microsoft Foundry uses projects to organize models, resources, data, and other assets used to develop an AI solution.

28. In a web browser, open the [Microsoft Foundry portal](https://ai.azure.com) at `https://ai.azure.com` to start building; signing in using your Azure credentials. Close any tips or quick start panes that are opened the first time you sign in.
29. If it is not already enabled, in the tool bar at the top of the page, enable the **New Foundry** option. Then, if prompted, create a new project with a unique name; expanding the **Advanced options** area to specify the following settings for your project:
- Foundry resource: Use the default name for your resource (usually {project_name}-resource)
 - Subscription: Your Azure subscription
 - Resource group: Create or select a resource group
 - Region: Select any of the AI Foundry recommended regions in [this list](https://learn.microsoft.com/azure/foundry/openai/how-to/responses#region-availability){:target="_blank"}
30. Wait for your project to be created. Then view its home page.

Explore models in the catalog

Microsoft Foundry Models provides a catalog of models that you can use in your project. You can browse the catalog and compare models to find the right one for your needs.

31. Now you're ready to explore models. On the **Discover** page, select the **Models** tab to view the Microsoft Foundry model catalog.

The model catalog lists all models available in Foundry. Some are provided directly from Azure (and billed through your Azure subscription) while others are provided by partners and the community.

Note that you can search and filter the catalog, based on model names, capabilities, and other factors.

32. Search for `gpt-5.2`. Then, in the search results, select the **gpt-5.2** model to view its **model card**. Model cards provide information about models to help you determine if they are suitable for your needs.
33. Read the description and review the other information available on the **Details** page.
34. View the **Benchmarks** page for the gpt-5.2 model to see how the model compares across some standard performance benchmarks with other models that are used in similar scenarios.
35. Use the back arrow (**←**) next to the **gpt-5.2** page title to return to the model catalog.

Compare models using the model leaderboard

Now let's use the model leaderboard and side-by-side comparison features to compare models visually.

36. In the model catalog page, select **View leaderboard**.
37. In the **Model leaderboard** page, review the top models ranked by quality, safety, cost, and performance. Note which models score highest for AI quality metrics.
38. Scroll down to use the **Trade-off chart** section to compare models on multiple dimensions.
39. Select the **Benchmark Cost** from the dropdown to see how model quality relates to cost, and then use the model list to compare **gpt-5.2** and **gpt-5-mini**. If you want to explore further, you can add other models to the comparison.
40. Select the **Throughput** metric from the dropdown to see how the quality of these models relates to throughput scores.

41. Select the **Safety** metric from the dropdown to see how the quality of these models relates to safety scores.
42. In the table just above the trade-off charts, you can compare benchmarks. Select **gpt-5.2** and **gpt-5-mini**, and optionally any other models you want to explore, and then use the **Compare models** button to view their benchmarks side-by-side.
43. Review the comparison across the following data:
 - Performance benchmarks: Quality, safety, and throughput scores.
 - Input and output: The formats supported for prompts and responses.
 - Context: The number of tokens that can be maintained in a conversation and produced as output, and when the model was trained.
 - Endpoints: The API endpoints through which the model can be consumed by client applications, and whether it can be used by an agent.
 - Supported features: Specific capabilities that you may require in your application scenario.
44. Use the back arrow (**←**) next to the **gpt-5.2** page title to return to the model catalog.

Deploy models

Now let's deploy the models we'll use for testing and evaluation. You need to deploy gpt-5.2 and gpt-5-mini.

Deploy the gpt-5.2 model

45. In the model catalog, search for `gpt-5.2` and select it.
46. On the model page, select **Deploy** and deploy the model using the *default settings*.

The deployed model will open in the model playground, where it will be selected in the Model drop-down list.

47. Note the deployment name that is assigned to the **gpt-5.2** model. You'll need to identify this deployment later.

Deploy the gpt-5-mini model

48. In the model playground, in the **Model** list, select **Browse more models**.
49. Search for `gpt-5-mini`, and then select it and deploy it.

The model is deployed and selected in the model playground.

50. Note the deployment name that is assigned to the **gpt-5-mini** model.

Compare models in the model playground

Now that you have two model deployments, let's compare them in the playground.

51. In the playground, ensure the deployment for the **gpt-5-mini** model is selected in the **Models** list, and then on the right side of the page, in the **Compare models** list, select the deployment for the **gpt-5.2** model.
52. The side-by-side comparison view opens directly into separate chat panes for each model. Select the **Chat** tab for both models, and enter the following prompt:

I have a fox, a chicken, and a bag of grain that I need to take over a river in a boat. I can only take one thing at a time. If I leave the chicken and the grain unattended, the chicken will eat the grain. If I leave the fox and the chicken unattended, the fox will eat the chicken. How can I get all three things across the river without anything being eaten?

53. Submit the prompt and view the responses from both models. Then, enter the following follow-up prompt:

Explain your reasoning.

54. Compare the responses from each model. Note any differences in accuracy, reasoning quality, and response style.

Evaluate a model with a synthetic dataset

The model playground is useful for quick manual testing, but to systematically assess a model's performance across many inputs, you can run an evaluation. Let's evaluate the gpt-5.2 model using a synthetically generated dataset of travel-related questions.

Step 1: Target

55. In the playground, select the **Evaluations** tab.
56. Select **Create** to open the **Create new evaluation** wizard.
57. For the evaluation target, select **Model**.
58. In the table of models, deselect any preselected deployments so that only the checkbox for **gpt-5.2** is selected, and then select **Next**.

Step 2: Data

Instead of uploading a test dataset, you'll use Foundry's synthetic data generation feature to create one automatically.

59. In the **Data** step, under **Dataset source**, select **Synthetic generation**.

With synthetic generation, a deployment is used to automatically generate questions for each target when you submit the evaluation.

60. Select **Generate**, and then set and confirm the following:

- Name of the new dataset: Leave as default
- Model: gpt-5.2
- Number of rows: 45
- Prompt: Create various travel related questions, and include some content safety and security tests
- Seed data: Leave blank

61. Select **Next** to proceed.

Step 3: Configure models

62. In the **Configure models** step, set the **Developer** prompt for the model being evaluated:

You are a helpful travel assistant that provides accurate, detailed, and practical travel advice to help users plan their trips.

63. Leave the rest of the values at their default, then select **Next**.

Step 4: Criteria

64. In the **Criteria** step, view all of the suggested evaluators. These use an AI model as a judge to assess the quality of responses.

65. Remove all of the criteria under **Agents** and **Safety**, leaving the rest of the evaluators enabled.

66. Select **Next**.

Step 5: Review and submit

67. In the **Review** step, verify the evaluation configuration, including the target model, dataset, and selected criteria.

68. Provide a name for the evaluation, such as `travel-assistant-eval`.

69. Select **Submit** to start the evaluation run.

70. Wait for the evaluation to complete. This may take several minutes, depending on data center load.

Review the results

71. When the evaluation completes, select the evaluation run to view the results page displays an overview of the evaluation metrics.

72. Review the scores and results from each evaluation in the table detailed on the run page. Scroll to the right and view additional pages, where you'll see mostly passing values. Depending on the model's response, you may see some failures. If you do, examine those closely.

73. Select the **Analyze results** button, selecting **gpt-5.2** from dropdown, then select **Start analysis**.

74. On this page you'll see any failures clustered by why they failed, where you can see details on why it failed. Most of those failures will be due to the model saying it's unable to help due to the nature of the question, however you should explore each failure and consider if the response is what you want to see.

75. Review any failures and the AI suggestions for how to improve. This guidance will help you tweak your configuration to perform better.

Clean up

If you've finished exploring Microsoft Foundry, you should delete the resources you have created in this exercise to avoid incurring unnecessary Azure costs.

76. Open the [Azure portal](https://portal.azure.com) and view the contents of the resource group where you deployed the resources used in this exercise.

77. On the toolbar, select **Delete resource group**.

78. Enter the resource group name and confirm that you want to delete it.

Acceptance Evidence

Complete the pinned Microsoft Learning exercise `official/mslearn-ai-studio/Instructions/Exercises/02-model-catalog-evaluation.md`; capture its expected output, relevant version/endpoint, one measured result, and an explained failure or limitation.

Troubleshooting Checklist

79. Capture the full error and request/operation ID before retrying.
80. Confirm the endpoint, API/model version, region, identity, and RBAC scope.
81. Validate payload/schema fields and input content type.
82. Check quota, rate limits, network resolution, private connectivity, and service status.
83. Record the corrective action and rerun the acceptance check.

Lab 03 - Build a generative chat app

Source folder: `labs/lab-03-build-a-generative-chat-app`

Objectives

- Complete the exact Microsoft Learning exercise for the Generative AI and agentic domain.
- Capture a working result and explain its product-development implications.

Source

The complete, unchanged Microsoft exercise and assets are in `[official/mslearn-ai-studio/Instructions/Exercises/03-foundry-sdk.md](../official/mslearn-ai-studio/Instructions/Exercises/03-foundry-sdk.md)`. The source revision and SHA-256 hashes are recorded in `[OFFICIAL-SOURCE-MANIFEST.json](../OFFICIAL-SOURCE-MANIFEST.json)`. Read the source file for every numbered instruction; this course wrapper does not alter those steps.

Steps

Prepare

Read the source exercise prerequisites and use a trainer-provided Azure subscription or approved lab environment. Do not commit your own `.env` values.

Complete the official exercise

Follow the numbered steps in `[03-foundry-sdk.md](../official/mslearn-ai-studio/Instructions/Exercises/03-foundry-sdk.md)` exactly. Use its companion `Labfiles` or `labfiles` directory in the same source snapshot.

Capture evidence

Record the deployment or resource names, model and API versions, input, output, latency or quality measure, and one failure or limitation. Remove personal data and live keys.

Validation

The official exercise completes with its expected output; a second learner can reproduce the result from the source instructions and the recorded environment.

Timing

Approximately 45 minutes for the official exercise; trainer discussion and WSQ evidence review are scheduled separately.

Unchanged Microsoft Learning exercise instructions

Exact Markdown source and companion assets:

labs/official/mslearn-ai-studio/Instructions/Exercises/03-foundry-sdk.md. Follow the numbered instructions below with the source assets. Microsoft Learning MIT license and pinned revision are in labs/official/ and the source manifest.

In this exercise, you use the OpenAI SDK and the Responses API to create a chat app that connects to a model deployed in a Microsoft Foundry project.

This exercise takes approximately 45 minutes.

> Note: Some of the technologies used in this exercise are in preview or in active development. You may experience some unexpected behavior, warnings, or errors.

Prerequisites

Before starting this exercise, ensure you have:

- An active [Azure subscription](https://azure.microsoft.com/pricing/purchase-options/azure-account)
- [Visual Studio Code](https://code.visualstudio.com/) installed
- [Python version **3.13.xx**](https://www.python.org/downloads/release/python-31312/) installed*
- [Git](https://git-scm.com/install/) installed and configured
- [Azure CLI](https://learn.microsoft.com/cli/azure/install-azure-cli?view=azure-cli-latest) installed

> \ Python 3.14 is available, but some dependencies are not yet compiled for that release. The lab has been successfully tested with Python 3.13.12.

Create a Microsoft Foundry project

Microsoft Foundry uses projects to organize models, resources, data, and other assets used to develop an AI solution.

84. In a web browser, open the [Microsoft Foundry portal](https://ai.azure.com) at `https://ai.azure.com` to start building; signing in using your Azure credentials. Close any tips or quick start panes that are opened the first time you sign in.

85. If it is not already enabled, in the tool bar at the top of the page, enable the **New Foundry** option. Then, if prompted, create a new project with a unique name; expanding the **Advanced options** area to specify the following settings for your project:

- Foundry resource: Use the default name for your resource (usually {project_name}-resource)
- Subscription: Your Azure subscription
- Resource group: Create or select a resource group

- Region: Select any of the AI Foundry recommended regions in [this list](https://learn.microsoft.com/azure/foundry/openai/how-to/responses#region-availability){:target="_blank"}

86. Wait for your project to be created. Then view its home page.

Deploy a model

Next, let's deploy a model that you'll use in your chat application.

87. Now you're ready to explore models. On the **Discover** page, select the **Models** tab to view the Microsoft Foundry model catalog.
88. In the model catalog, search for `gpt-5.2``.
89. Review the model card, and then deploy it using the default settings.
90. When the model has been deployed, it will open in the model playground - you can test it there if you like.

Get the endpoint

You'll need an endpoint to connect to the model from a client application. In this exercise, we're going to use the OpenAI SDK to chat with the model; and we'll use the Azure OpenAI endpoint with Entra ID authentication to connect to it.

> Note: As an alternative to Entra ID authentication, you could use the API Key for the project. using Entra ID authentication is preferred whenever possible.

91. On the menu bar, select the **Home** page.
92. Note the **Azure OpenAI Endpoint** displayed there.

> Tip: You'll use the Azure OpenAI Endpoint in this exercise, not the project endpoint!

Create a client application to chat with the model

Now that you have deployed a model, you can use the OpenAI SDK and the Responses API to develop an application that chats with it.

Get the application files from GitHub

The initial application files you'll need to develop your chat application are provided in a GitHub repo.

93. Open Visual Studio Code.
94. Open the command palette (`*Ctrl+Shift+P*`) and use the ``Git:clone`` command to clone the ``https://github.com/microsoftlearning/mslearn-ai-studio`` repo to a local folder (it doesn't matter which one). Then open it.

You may be prompted to confirm you trust the authors.

Prepare the application configuration

95. In Visual Studio Code, view the **Extensions** pane; and if it is not already installed, install the **Python** extension.
96. In the **Command Palette**, use the command ``python:select interpreter``. Then create a new **Venv** environment based on your Python 3.13 installation.

> Tip: If you are prompted to install dependencies, you can install the ones in the `requirements.txt` file in the `/labfiles/foundry-chat/python/chat-app` folder; but it's OK if you don't - we'll install them later!

97. In the Explorer pane, navigate to the folder containing the application code files at `**/labfiles/foundry-chat/python/chat-app**`. The application files include:

- `.env` (the application configuration file)
- `requirements.txt` (the Python package dependencies that need to be installed)
- `chat-app.py` (the code file for the chat application)
- `chat-async.py` (the code file for an asynchronous version of the application)

98. In the **Explorer** pane, right-click the **chat-app** folder containing the application files, and select **Open in integrated terminal** (or open a terminal in the **Terminal** menu and navigate to the `/labfiles/foundry-chat/python/chat-app` folder.)

> Note: Opening the terminal in Visual Studio Code will automatically activate the Python environment. You may need to enable running scripts on your system.

99. Ensure that the terminal is open in the `labfiles/foundry-chat/python/chat-app` folder with the prefix `(.venv)` to indicate that the Python environment you created is active.

100. Install the OpenAI SDK, Azure Identity, and other required packages by running the following command:

```
pip install -r requirements.txt
```

101. In the **Explorer** pane, in the `labfiles/foundry-chat/python/chat-app` folder, select the `.env` file to open it. Then update the configuration values to include the **Azure OpenAI Endpoint** and the name assigned to the deployment for the `gpt-5.2` model.

> Tip: Copy the Azure OpenAI Endpoint (not the project endpoint!) from the project home page in the Foundry portal, and enter the exact deployment name assigned to your deployment in the `MODEL_DEPLOYMENT` setting.

Save the modified configuration file.

Use the **ChatCompletions** API to chat with the model

The ChatCompletions API is a well-established way to build client applications for large language models, and has been widely adopted.

102. In the **Explorer** pane, in the `labfiles/foundry-chat/python/chat-app` folder, select the `chat-app.py` file (not `chat-async.py`) to open it.

103. Review the existing code. You will add code to use the OpenAI SDK to access your model.

> Tip: As you add code to the code file, be sure to maintain the correct indentation.

104. At the top of the code file, under the existing namespace references, find the comment `Import namespaces` and add the following code to import the namespace you will need to use the OpenAI SDK:

```
python

# import namespaces

from openai import OpenAI

from azure.identity import DefaultAzureCredential, get_bearer_token_provider
```

105. In the `main` function, note that code to load the endpoint and key from the configuration file has already been provided. Then find the comment `Initialize the OpenAI client`, and add the following code to create a client for the OpenAI API:

```
python

# Initialize the OpenAI client

token_provider = get_bearer_token_provider(
    DefaultAzureCredential(), "https://ai.azure.com/.default"
)

openai_client = OpenAI(
    base_url=azure_openai_endpoint,
    api_key=token_provider
)
```

106. In the ****main**** function, note that code to request a user prompt until the user quits the app has been provided. Within this loop, find the ****Get a response**** comment, and add the following code:

```
python

# Get a response

completion = openai_client.chat.completions.create(
    model=model_deployment,
    messages=[
        {
            "role": "system",
            "content": "You are a helpful AI assistant that answers questions and provides
information."
        },
        {
            "role": "user",
            "content": input_text
        }
    ]
)

print(completion.choices[0].message.content)
```

Note that the ChatCompletions API uses a JSON collection of messages to encapsulate the conversation. Often, these consist of a system prompt that provides instructions to the model, and a user prompt that includes the user's input.

107. Save the changes to the code file. Then, in the terminal pane, use the following command to sign into Azure.

```
powershell  
  
az login
```

> Note: In most scenarios, just using `az login` will be sufficient. However, if you have subscriptions in multiple tenants, you may need to specify the tenant by using the `--tenant` parameter. See [Sign into Azure interactively using the Azure CLI](<https://learn.microsoft.com/cli/azure/authenticate-azure-cli-interactively>) for details.

108. When prompted, follow the instructions to sign into Azure. Then complete the sign in process in the command line, viewing (and confirming if necessary) the details of the subscription containing your Foundry resource.
109. After you have signed in, enter the following command to run the application:

```
powershell  
  
python chat-app.py
```

The program should run in the terminal (if not, resolve any errors and try again).

110. When prompted, enter the following prompt:

```
input  
  
Tell me about the ELIZA chatbot.
```

After a few moments, the app should respond with some information about the ELIZA chatbot created in the 1960s.

111. Enter the prompt `'quit'` to end the application.

Use the ***Responses*** API to chat with the model

While the ChatCompletions API is widely used, it is increasingly being superseded by the newer Responses API. Let's update the code to use it.

112. In the `**chat-app.py**` code, in the `**main**` function, replace the code under the comment `**Get a response**` with the following code that uses the ***Responses*** API.

```
python  
  
# Get a response  
  
response = openai_client.responses.create(
```

```

        model=model_deployment,

        instructions="You are a helpful AI assistant that answers questions and provides
information.",

        input=input_text

    )

    print(response.output_text)

```

Note the simpler syntax in which the system message is assigned to the instructions parameter, and the user prompt is assigned to the input parameter.

113. Save the changes to the code, and in the terminal pane, re-run the application (`python chat-app.py`).

114. When prompted, enter the same prompt as before:

```

input

Tell me about the ELIZA chatbot.

```

After a few moments, the app should once again respond with some information about the ELIZA chatbot.

115. Enter the following prompt to try to continue the conversation:

```

input

How does it compare to modern LLMs?

```

The app should respond in a way that indicates it doesn't understand what "it" refers to. The conversation context has been lost. We'll fix that.

116. Enter the prompt ``quit`` to end the application.

Add conversation tracking

To maintain the conversational context, we need to include references to previous responses in each new request.

117. In the `chat-app.py` code, in the `main` function, find the comment `Loop until the user wants to quit`, and add the following code above it (*before* the loop):

```

python

# Track responses

last_response_id = None

```

118. Modify the code under the comment ****Get a response**** with the following code to pass the previous response ID on the request, and then obtain the new response ID so it can be added next time.

```
python

# Get a response

response = openai_client.responses.create(

    model=model_deployment,

    instructions="You are a helpful AI assistant that answers questions and provides
information.",

    input=input_text,

    previous_response_id=last_response_id,

)

print(response.output_text)

last_response_id = response.id
```

Using this technique, you can pass the ID of the previous response to maintain context. You could also implement more complex logic to pass an ID from any previous response to redirect a conversation or resume a previous conversational thread.

119. Save the changes to the code, and in the terminal pane, re-run the application (`python chat-app.py`).

120. When prompted, enter the same prompt as before:

```
input
```

```
Tell me about the ELIZA chatbot.
```

After a few moments, the app should once again respond with some information about the ELIZA chatbot.

121. Enter the following prompt to try to continue the conversation:

```
input
```

```
How does it compare to modern LLMs?
```

This time, the app should respond with a comparison of the ELIZA chatbot and modern LLMs. The response may be quite lengthy, and the app waits until it has all been received from the model before displaying it, which may make the app seem unresponsive. We'll fix that next!

122. Enter the prompt ``quit`` to end the application.

Implement **streaming** responses

To handle long responses, you can use streaming to start processing partial responses before the full text has been returned.

123. In the `chat-app.py` code, in the `main` function, replace the code under the comment `Get a response` with the following code that uses **streaming**.

```
python

# Get a response

stream = openai_client.responses.create(
    model=model_deployment,
    instructions="You are a helpful AI assistant that answers questions and provides
information.",
    input=input_text,
    previous_response_id=last_response_id,
    stream=True
)

for event in stream:
    if event.type == "response.output_text.delta":
        print(event.delta, end="")
    elif event.type == "response.completed":
        last_response_id = event.response.id
print()
```

Note that the `stream=True` parameter creates a streamed response in which events occur as each new chunk (or delta) is ready for processing.

124. Save the changes to the code, and in the terminal pane, re-run the application (`python chat-app.py`).

125. When prompted, enter the same prompt as before:

```
input

Tell me about the ELIZA chatbot.
```

After a few moments, the app should start responding with some information about the ELIZA chatbot. The response should appear incrementally as each chunk is returned.

126. Enter the following prompt to try to continue the conversation:

input

How does it compare to modern LLMs?

Again, the response should be displayed incrementally.

127. Enter the prompt `quit` to end the application.

Use the asynchronous API

The OpenAI SDK offers an asynchronous option that can increase the responsiveness of applications when using long-running model or agent operations.

128. In the **Explorer** pane, in the **labfiles/foundry-chat/python/chat-app** folder, select the **chat-async.py** file (not **chat-app.py**) to open it.

129. Review the existing code. You will add code to use the OpenAI SDK async API to access your model.

> Tip: As you add code to the code file, be sure to maintain the correct indentation.

130. At the top of the code file, under the existing namespace references, find the comment **Import namespaces** and add the following code to import the namespace you will need to use the OpenAI SDK:

```
python

# import namespaces for async

import asyncio

from openai import AsyncOpenAI

from azure.identity.aio import DefaultAzureCredential, get_bearer_token_provider
```

131. In the **main** function, note that code to load the endpoint and key from the configuration file has already been provided. Then find the comment **Initialize an async OpenAI client**, and add the following code to create a client for the OpenAI API:

```
python

# Initialize an async OpenAI client

credential = DefaultAzureCredential()

token_provider = get_bearer_token_provider(

    credential, "https://ai.azure.com/.default"

)

async_client = AsyncOpenAI(

    base_url=azure_openai_endpoint,

    api_key=token_provider
```

)

132. In the ****main**** function, note that code to request a user prompt until the user quits the app has been provided. Within this loop, find the ****Await an asynchronous response**** comment, and add the following code:

```
python

# Await an asynchronous response

response = await async_client.responses.create(

    model=model_deployment,

    instructions="You are a helpful AI assistant that answers questions and provides
information.",

    input=input_text,

    previous_response_id=last_response_id

)

assistant_text = response.output_text

print("Assistant:", assistant_text)

last_response_id = response.id
```

This code awaits an asynchronous response from the model.

133. At the end of the ****main**** function, in the ****finally**** block, find the comment ****Close the async client session**** and add the following code to close the asynchronous client:

```
python

# Close the async client session

await credential.close()
```

134. Save the changes to the code file. Then, in the terminal pane, use the following command to run the program:

```
powershell

python chat-async.py
```

The program should run in the terminal (if not, resolve any errors and try again).

135. When prompted, enter the following prompt:

input

Tell me about the Turing test.

After a few moments, the app should respond with some information about the Turing test.

136. Enter the prompt `quit` to end the application.

Summary

In this exercise, you used the OpenAI SDK and the ChatCompletions and Responses APIs to create a client application for a generative AI model that you deployed in a Microsoft Foundry project. You customized the model's behavior by tracking conversational context and implemented streaming to deliver a responsive chat experience.

Clean up

If you've finished exploring Microsoft Foundry, you should delete the resources you have created in this exercise to avoid incurring unnecessary Azure costs.

137. Open the [Azure portal](https://portal.azure.com) and view the contents of the resource group where you deployed the resources used in this exercise.

138. On the toolbar, select **Delete resource group**.

139. Enter the resource group name and confirm that you want to delete it.

Acceptance Evidence

Complete the pinned Microsoft Learning exercise [official/mslearn-ai-studio/Instructions/Exercises/03-foundry-sdk.md](https://mslearn-ai-studio/Instructions/Exercises/03-foundry-sdk.md); capture its expected output, relevant version/endpoint, one measured result, and an explained failure or limitation.

Troubleshooting Checklist

140. Capture the full error and request/operation ID before retrying.

141. Confirm the endpoint, API/model version, region, identity, and RBAC scope.

142. Validate payload/schema fields and input content type.

143. Check quota, rate limits, network resolution, private connectivity, and service status.

144. Record the corrective action and rerun the acceptance check.

Lab 04 - Build a Foundry agent

Source folder: labs/lab-04-build-a-foundry-agent

Objectives

- Complete the exact Microsoft Learning exercise for the Generative AI and agentic domain.
- Capture a working result and explain its product-development implications.

Source

The complete, unchanged Microsoft exercise and assets are in [official/mslearn-ai-agents/Instructions/Exercises/01-build-agent-portal-and-vscode.md](../official/mslearn-ai-agents/Instructions/Exercises/01-build-agent-portal-and-vscode.md). The source revision and SHA-256 hashes are recorded in [OFFICIAL-SOURCE-MANIFEST.json](../OFFICIAL-SOURCE-MANIFEST.json). Read the source file for every numbered instruction; this course wrapper does not alter those steps.

Steps

Prepare

Read the source exercise prerequisites and use a trainer-provided Azure subscription or approved lab environment. Do not commit your own .env values.

Complete the official exercise

Follow the numbered steps in [01-build-agent-portal-and-vscode.md](../official/mslearn-ai-agents/Instructions/Exercises/01-build-agent-portal-and-vscode.md) exactly. Use its companion Labfiles or labfiles directory in the same source snapshot.

Capture evidence

Record the deployment or resource names, model and API versions, input, output, latency or quality measure, and one failure or limitation. Remove personal data and live keys.

Validation

The official exercise completes with its expected output; a second learner can reproduce the result from the source instructions and the recorded environment.

Timing

Approximately 45 minutes for the official exercise; trainer discussion and WSQ evidence review are scheduled separately.

Unchanged MicrosoftLearning exercise instructions

Exact Markdown source and companion assets:

labs/official/mslearn-ai-agents/Instructions/Exercises/01-build-agent-portal-and-vscode.md. Follow the numbered instructions below with the source assets. MicrosoftLearning MIT license and pinned revision are in labs/official/ and the source manifest.

In this exercise, you'll build a complete AI agent solution using both the Microsoft Foundry portal and the Foundry Toolkit VS Code extension. You'll start by creating a basic agent in the portal with grounding data and built-in tools, then interact with it programmatically using VS Code to use advanced capabilities like code interpreter for data analysis.

This exercise takes approximately 45 minutes.

> Note: Some of the technologies used in this exercise are in preview or in active development. You may experience some unexpected behavior, warnings, or errors.

Prerequisites

Before starting this exercise, ensure you have:

- An [Azure subscription](https://azure.microsoft.com/free/) with sufficient permissions and quota to provision Azure AI resources
- [Visual Studio Code](https://code.visualstudio.com/) installed on your local machine
- [Python 3.13](https://www.python.org/downloads/) installed
- [Git](https://git-scm.com/downloads) installed on your local machine
- Basic familiarity with Azure AI services and Python programming

> \ Python 3.14 isn't supported yet: some dependencies have no 3.14 build. This lab was tested with Python 3.13.12.

Create a Microsoft Foundry Project

Microsoft Foundry uses projects to organize models, resources, data, and other assets used to develop an AI solution.

145. In a web browser, open the [Foundry portal](https://ai.azure.com) at `https://ai.azure.com` and sign in using your Azure credentials. Close any tips or quick start panes that are opened the first time you sign in, and if necessary use the **Foundry** logo at the top left to navigate to the home page.

> Important: For this lab, you're using the New Foundry experience.

146. In the top banner, select **Start building** to try the new Microsoft Foundry Experience.

147. When prompted, create a **new** project, and enter a valid name for your project (e.g., `it-support-agent-project`).

148. Expand **Advanced options** and specify the following settings:

- Microsoft Foundry resource: A valid name for your Foundry resource
- Region: Select one available near you\
- Subscription: Your Azure subscription
- Resource group: Select your resource group, or create a new one

> \ Some Azure AI resources are constrained by regional model quotas. In the event of a quota limit being exceeded later in the exercise, there's a possibility you may need to create another resource in a different region.

149. Select **Create** and wait for your project to be created.

150. When your project is created, a welcome dialog may appear. Select **Next** to read through the welcome message, and then select **Create agent**.

You can also select Start building on the home page, and select Create agents from the drop-down menu.

151. Set the **Agent name** to `it-support-agent` and create the agent.

The playground will open for your newly created agent. You'll see that an available deployed model is already selected for you.

Configure your agent with instructions and grounding data

Now that you have an agent created, let's configure it with instructions and add grounding data.

152. In the agent playground, set the **Instructions** to:

prompt

You are an IT Support Agent for Contoso Corporation.

You help employees with technical issues and IT policy questions.

Guidelines:

- Always be professional and helpful
- Use the IT policy documentation to answer questions accurately

- If you don't know the answer, admit it and suggest contacting IT support directly
- When creating tickets, collect all necessary information before proceeding

153. Download the IT policy document from the lab repository. Open a new browser tab and navigate to:

https://raw.githubusercontent.com/MicrosoftLearning/mslearn-ai-agents/main/Labfiles/01-build-agent-portal-and-vscode/IT_Policy.txt

Save the file to your local machine.

> Note: This document contains sample IT policies for password resets, software installation requests, and hardware troubleshooting.

154. Return to the agent playground. In the **Tools** section, select **Add**, and then add both **File search** and **Code interpreter**.

155. To the right of **Add**, select **Upload files**. Under **Attach files**, browse to and upload the `IT_Policy.txt` file you just downloaded, and then select **Attach**.

156. Wait for the file to be indexed. You'll see a confirmation when it's ready.

157. Now let's add some performance data for the code interpreter to analyze. Download the system performance data file from:

https://raw.githubusercontent.com/MicrosoftLearning/mslearn-ai-agents/main/Labfiles/01-build-agent-portal-and-vscode/system_performance.csv

Save this file to your local machine.

158. To the right of **Code interpreter**, select **+ Files**, and then upload the `system_performance.csv` file you just downloaded.

> Note: This CSV file contains simulated system metrics (CPU, memory, disk usage) over time that the agent can analyze.

159. Save the agent.

Test your agent

Let's test the agent to see how it responds using the grounding data.

160. In the chat interface on the right side of the playground, enter the following prompt:

What's the policy for password resets?

161. Review the response. The agent should reference the IT policy document and provide accurate information about password reset procedures.
162. Try another prompt:

How do I request new software?

163. Again, review the response and observe how the agent uses the grounding data.
164. Now test the code interpreter with a data analysis request:

Can you analyze the system performance data and tell me if there are any concerning trends?

165. The agent should use the code interpreter to analyze the CSV file and provide insights about system performance.
166. Try asking for a visualization:

Create a chart showing CPU usage over time from the performance data

167. The agent will use code interpreter to generate visualizations and analysis.

Great! You've created an agent with grounding data, file search, and code interpreter capabilities. In the next section, you'll interact with this agent programmatically using VS Code.

Interact with your agent using VS Code

As a developer, you may spend some time working in the Foundry portal; but you're also likely to spend a lot of time in Visual Studio Code. The Foundry Toolkit for VS Code extension provides a convenient way to work with Foundry project resources without leaving the development environment.

Install and configure the VS Code extension

If you already have installed the Foundry Toolkit extension, you can skip this section.

168. Open Visual Studio Code.
169. Select **Extensions** from the left pane (or press **Ctrl+Shift+X**).
170. Search the extensions marketplace for the 'Foundry Toolkit for VS Code' extension from Microsoft and select **Install**.

Installing the Foundry Toolkit Extension will add the Foundry Toolkit extension to VS Code.

> Note: The extension is currently listed as Foundry Toolkit, but some VS Code labels, commands, or older screenshots may still refer to AI Toolkit. In this lab, treat those names as referring to the same extension experience.

171. After installing the extension, select the Foundry Toolkit icon in the sidebar.

You should be prompted to sign in to your Azure account if you haven't already.

Test your agent in VS Code

Before writing any code, you can interact with your agent directly in the extension interface.

172. Under **Microsoft Foundry Resources**, choose **Set Default Project**

If a default project is already active, the project name will appear in the resources list. You can select a different project by selecting the same Select project icon.

173. Expand the project section. Under **Prompt Agents**, you should see the `it-support-agent` you created in the portal. Select the agent name to open the Agent Builder interface.

The agent playground will appear in the Agent Builder interface, allowing you to interact with the agent and configure its settings without leaving VS Code.

174. In the playground chat pane, type a question such as:

What is the policy for reporting a lost or stolen device?

175. Review the agent's response. It should use the grounding data you uploaded earlier to provide relevant IT policy information.

> Tip: You can use this built-in playground to quickly test your agent's instructions and knowledge without writing any code.

Create a client application to interact with your agent

Now let's create a client application that interacts with your agent programmatically.

176. In VS Code, open the Command Palette (**Ctrl+Shift+P** or **View > Command Palette**).
177. Type **Git: Clone** and select it from the list.
178. Enter the repository URL:

<https://github.com/MicrosoftLearning/mslearn-ai-agents.git>

179. Choose a location on your local machine to clone the repository.
180. When prompted, select **Open** to open the cloned repository in VS Code.
181. Once the repository opens, select **File > Open Folder** and navigate to `mslearn-ai-agents/Labfiles/01-build-agent-portal-and-vscode/Python`, then choose **Select Folder**.
182. In the Explorer pane, open the `agent_with_functions.py` file. If the file is empty, replace its contents with the following code.
183. Use the following code:

```
python
import base64

import os
```

```

from pathlib import Path

from azure.ai.projects import AIProjectClient

from azure.identity import DefaultAzureCredential

from dotenv import load_dotenv

OUTPUT_DIR = Path("agent_outputs")

def get_output_path(filename):
    """Create a unique path for generated files."""
    OUTPUT_DIR.mkdir(exist_ok=True)

    file_name = Path(filename).name

    stem = Path(file_name).stem or "output"

    suffix = Path(file_name).suffix

    output_path = OUTPUT_DIR / file_name

    counter = 1

    while output_path.exists():
        output_path = OUTPUT_DIR / f"{stem}_{counter}{suffix}"
        counter += 1

    return output_path

def save_bytes(file_bytes, filename):
    """Save binary content to a local file."""
    output_path = get_output_path(filename)

    with open(output_path, "wb") as file_handle:
        file_handle.write(file_bytes)

    return output_path

def save_image(image_data, filename):
    """Save base64 image data to a file."""
    return save_bytes(base64.b64decode(image_data), filename)

def download_container_file(openai_client, annotation, downloaded_files):
    """Download a cited container file once and return its local path."""
    cache_key = (annotation.container_id, annotation.file_id)

    if cache_key in downloaded_files:

```

```

        return downloaded_files[cache_key]

file_content = openai_client.containers.files.content.retrieve(
    file_id=annotation.file_id,
    container_id=annotation.container_id,
)

output_path = save_bytes(
    file_content.read(),
    annotation.filename or f"{annotation.file_id}.bin",
)

downloaded_files[cache_key] = output_path

return output_path

def format_output_text(content_item, openai_client, downloaded_files):
    """Replace sandbox file citations with local file paths."""
    text = content_item.text or ""
    replacements = []
    referenced_files = set()

    for annotation in content_item.annotations or []:
        if getattr(annotation, "type", "") != "container_file_citation":
            continue

        output_path = download_container_file(openai_client, annotation, downloaded_files)
        replacement_text = f"{annotation.filename} (saved to {output_path})"
        referenced_files.add(output_path)

        start_index = getattr(annotation, "start_index", None)
        end_index = getattr(annotation, "end_index", None)

        if start_index is not None and end_index is not None:
            replacements.append((start_index, end_index, replacement_text))
            continue

        annotated_text = getattr(annotation, "text", "")

        if annotated_text:
            text = text.replace(annotated_text, replacement_text)

```

```

for start_index, end_index, replacement_text in sorted(replacements, reverse=True):
    text = f"{text[:start_index]}{replacement_text}{text[end_index:]}"

return text, referenced_files

def main():
    # Initialize the project client
    load_dotenv()

    project_endpoint = os.environ.get("PROJECT_ENDPOINT")
    agent_name = os.environ.get("AGENT_NAME", "it-support-agent")

    if not project_endpoint:
        print("Error: PROJECT_ENDPOINT environment variable not set")
        print("Please set it in your .env file or environment")
        return

    print("Connecting to Microsoft Foundry project...")
    credential = DefaultAzureCredential()
    project_client = AIProjectClient(
        credential=credential,
        endpoint=project_endpoint
    )

    # Get the OpenAI client for Responses API
    openai_client = project_client.get_openai_client()

    # Get the agent created in the portal
    print(f"Loading agent: {agent_name}")
    agent = project_client.agents.get(agent_name=agent_name)
    print(f"Connected to agent: {agent.name} (id: {agent.id})")

    # Create a conversation
    conversation = openai_client.conversations.create(items=[])
    print(f"Conversation created (id: {conversation.id})")

    # Chat loop
    print("\n" + "="*60)

    print("IT Support Agent Ready!")

```

```

print("Ask questions, request data analysis, or get help.")
print("Type 'exit' to quit.")
print("=60 + "\n")
while True:
    user_input = input("You: ").strip()
    if user_input.lower() in ['exit', 'quit', 'bye']:
        print("Goodbye!")
        break
    if not user_input:
        continue
    # Add user message to conversation
    openai_client.conversations.items.create(
        conversation_id=conversation.id,
        items=[{"type": "message", "role": "user", "content": user_input}]
    )
    # Get response from agent
    print("\n[Agent is thinking...]")
    response = openai_client.responses.create(
        conversation=conversation.id,
        extra_body={"agent_reference": {"name": agent.name, "type": "agent_reference"}},
        input=""
    )
    # Display response and save any generated files locally
    handled_output = False
    downloaded_files = {}
    referenced_files = set()
    image_count = 0
    if hasattr(response, "output") and response.output:
        for item in response.output:
            item_type = getattr(item, "type", "")

```

```

if item_type == "message" and getattr(item, "content", None):
    for content_item in item.content:
        if getattr(content_item, "type", "") != "output_text":
            continue

        formatted_text, message_files = format_output_text(
            content_item,
            openai_client,
            downloaded_files,
        )

        referenced_files.update(message_files)

        if formatted_text:
            print(f"\nAgent: {formatted_text}\n")
            handled_output = True

elif hasattr(item, "text") and item.text:
    print(f"\nAgent: {item.text}\n")
    handled_output = True

elif item_type == "image":
    image_count += 1
    filename = f"chart_{image_count}.png"

    if hasattr(item, "image") and hasattr(item.image, "data"):
        file_path = save_image(item.image.data, filename)
        print(f"\n[Agent generated a chart - saved to: {file_path}]")
    else:
        print("\n[Agent generated an image]")
        handled_output = True

for file_path in downloaded_files.values():
    if file_path not in referenced_files:
        print(f"\n[Agent generated a file - saved to: {file_path}]")
        handled_output = True

if not handled_output and hasattr(response, "output_text") and response.output_text:

```

```

        print(f"\nAgent: {response.output_text}\n")

if __name__ == "__main__":
    main()

```

184. Save the `agent\_with\_functions.py` file (**Ctrl+S** or **File > Save**).

Configure environment and run the application

185. In the Explorer pane, you'll see `.env.example` and `requirements.txt` files already present in the folder.
186. Duplicate the `.env.example` file, and rename it to `.env`.
187. In the `.env` file, replace `your\_project\_endpoint\_here` with your actual project endpoint:

```

PROJECT_ENDPOINT=<your_project_endpoint>

AGENT_NAME=it-support-agent

```

To get your project endpoint: In VS Code, open the Foundry Toolkit extension, right-click on your active project, and select Copy Endpoint. If Copy Endpoint isn't available in your installed version of Foundry Toolkit, open the Microsoft Foundry portal, go to your project, and copy the project endpoint from the project overview page instead.

188. Save the `.env` file (**Ctrl+S** or **File > Save**).
189. Open a terminal in VS Code (**Terminal > New Terminal**) and navigate to the working directory.
190. Install the required packages and login:

```

bash

python -m venv labenv

.\labenv\Scripts\Activate.ps1

pip install -r requirements.txt

```

```

bash

az login

```

191. Run the application:

```

bash

python agent_with_functions.py

```

Test the client application

When the agent starts, try these prompts to test different capabilities:

192. Test policy search with file search:

What's the policy for password resets?

193. Request data analysis with code interpreter:

Analyze the system performance data and identify any periods where CPU usage exceeded 80%

194. Request a visualization:

Create a line chart showing memory usage trends over time

The application saves generated charts and cited files to the agent_outputs folder and prints the local file path in the terminal.

195. Ask for statistical analysis:

What are the average, minimum, and maximum values for disk usage in the performance data?

196. Combined analysis:

Find any correlation between high CPU usage and memory usage in the performance data

Observe how the agent uses both file search (for policy questions) and code interpreter (for data analysis) to fulfill your requests. The code interpreter will analyze the CSV data, perform calculations, and can even generate visualizations. Type exit when done testing.

Cleanup

To avoid unnecessary Azure charges, delete the resources you created:

197. In the Foundry portal, navigate to your project

198. Select ****Settings**** > ****Delete project****

199. Alternatively, delete the entire resource group from the Azure portal

Acceptance Evidence

Complete the pinned Microsoft Learning exercise `official/mslearn-ai-agents/Instructions/Exercises/01-build-agent-portal-and-vscode.md`; capture its expected output, relevant version/endpoint, one measured result, and an explained failure or limitation.

Troubleshooting Checklist

200. Capture the full error and request/operation ID before retrying.
201. Confirm the endpoint, API/model version, region, identity, and RBAC scope.
202. Validate payload/schema fields and input content type.
203. Check quota, rate limits, network resolution, private connectivity, and service status.
204. Record the corrective action and rerun the acceptance check.

Lab 05 - Use a custom tool in an agent

Source folder: `labs/lab-05-use-a-custom-tool-in-an-agent`

Objectives

- Complete the exact Microsoft Learning exercise for the Generative AI and agentic domain.
- Capture a working result and explain its product-development implications.

Source

The complete, unchanged Microsoft exercise and assets are in `[official/mslearn-ai-agents/Instructions/Exercises/02-agent-custom-tools.md](../official/mslearn-ai-agents/Instructions/Exercises/02-agent-custom-tools.md)`. The source revision and SHA-256 hashes are recorded in `[OFFICIAL-SOURCE-MANIFEST.json](../OFFICIAL-SOURCE-MANIFEST.json)`. Read the source file for every numbered instruction; this course wrapper does not alter those steps.

Steps

Prepare

Read the source exercise prerequisites and use a trainer-provided Azure subscription or approved lab environment. Do not commit your own `.env` values.

Complete the official exercise

Follow the numbered steps in `[02-agent-custom-tools.md](../official/mslearn-ai-agents/Instructions/Exercises/02-agent-custom-tools.md)` exactly. Use its companion `Labfiles` or `labfiles` directory in the same source snapshot.

Capture evidence

Record the deployment or resource names, model and API versions, input, output, latency or quality measure, and one failure or limitation. Remove personal data and live keys.

Validation

The official exercise completes with its expected output; a second learner can reproduce the result from the source instructions and the recorded environment.

Timing

Approximately 50 minutes for the official exercise; trainer discussion and WSQ evidence review are scheduled separately.

Unchanged Microsoft Learning exercise instructions

Exact Markdown source and companion assets:

labs/official/mslearn-ai-agents/Instructions/Exercises/02-agent-custom-tools.md. Follow the numbered instructions below with the source assets. Microsoft Learning MIT license and pinned revision are in labs/official/ and the source manifest.

In this exercise you'll explore creating an agent that can use custom functions as a tool to complete tasks. The agent will act as an astronomy assistant that can provide information about astronomical events and calculate the cost of telescope rentals based on user inputs. You'll define the function tools and implement the logic to process function calls made by the agent.

> Tip: The code used in this exercise is based on the Microsoft Foundry SDK for Python. You can develop similar solutions using the SDKs for Microsoft .NET, JavaScript, and Java. Refer to [Microsoft Foundry SDK client libraries](https://learn.microsoft.com/azure/ai-foundry/how-to/develop/sdk-overview) for details.

This exercise should take approximately 50 minutes to complete.

> Note: Some of the technologies used in this exercise are in preview or in active development. You may experience some unexpected behavior, warnings, or errors.

Prerequisites

Before starting this exercise, ensure you have:

- [Visual Studio Code](https://code.visualstudio.com/) installed on your local machine
- An active [Azure subscription](https://azure.microsoft.com/free/)
- [Python 3.13](https://www.python.org/downloads/) installed
- [Git](https://git-scm.com/downloads) installed on your local machine

> \ Python 3.14 isn't supported yet: some dependencies have no 3.14 build. This lab was tested with Python 3.13.12.

Create a Foundry project with the Foundry Toolkit for VS Code extension

As a developer, you may spend some time working in the Foundry portal; but you're also likely to spend a lot of time in Visual Studio Code. The Foundry Toolkit for VS Code extension provides a convenient way to work with Foundry project resources without leaving the development environment.

205. Open Visual Studio Code.
206. Select **Extensions** from the left pane (or press **Ctrl+Shift+X**).
207. Search the extensions marketplace for the 'Foundry Toolkit for VS Code' extension from Microsoft and select **Install**.

Installing the Foundry Toolkit Extension will add the Foundry Toolkit extension to VS Code.

> Note: The extension is currently listed as Foundry Toolkit, but some VS Code labels, commands, or older screenshots may still refer to AI Toolkit. In this lab, treat those names as referring to the same extension experience.

208. After installing the extension, select the Foundry Toolkit icon in the sidebar.

You should be prompted to sign in to your Azure account if you haven't already.

209. Select **Create Project** under **Microsoft Foundry Resources**.

If a default project is already active, the project name will appear under My Resources. You can create a new project by right-clicking on the active project and selecting Switch Default Project in Azure Extension.

210. Select your Azure subscription and resource group, then enter a name for your Foundry project to create a new project for this exercise.

When the deployment is complete, you should see the project appear in the Foundry Toolkit pane as the default project.

Deploy a model

At the core of any generative AI project, there's at least one generative AI model. In this task, you'll deploy a model from the Model Catalog to use with your agent.

211. When the "Project deployed successfully" popup appears, select the **Deploy a new model** button. This opens the Model Catalog.

> Tip: You can also access the Model Catalog by selecting the + icon next to Models in the Resources section, or by pressing F1 and running the command Foundry Toolkit: Show model catalog.

212. In the Model Catalog, locate the **gpt-5** model (you can use the search bar to find it quickly).

213. Select **Deploy** next to the gpt-5 model.

214. Configure the deployment settings:

- Deployment name: Enter a name like "gpt-5"
- Deployment type: Select Global Standard (or Standard if Global Standard is not available)
- Model version: Leave as default
- Tokens per minute: Leave as default

215. Select **Deploy to Microsoft Foundry** in the bottom-left corner.

216. Wait for the deployment to complete. Your deployed model will appear under the **Models** section in the Resources view.

217. Right-click the name of the project deployment and select **Copy Project Endpoint**. You'll need this URL to connect your agent to the Foundry project in the next steps.

![Screenshot of copying the project endpoint in the Foundry Toolkit VS Code extension.](../Media/vs-code-endpoint.png)

Clone the starter code repository

For this exercise, you'll use starter code that will help you connect to your Foundry project and create an agent that uses custom function tools.

218. In VS Code, open the Command Palette (**Ctrl+Shift+P** or **View > Command Palette**).
219. Type **Git: Clone** and select it from the list.
220. Enter the repository URL:

<https://github.com/MicrosoftLearning/mslearn-ai-agents.git>

221. Choose a location on your local machine to clone the repository.
222. When prompted, select **Open** to open the cloned repository in VS Code.
223. Once the repository opens, select **File > Open Folder** and navigate to ``mslearn-ai-agents/Labfiles/02-agent-custom-tools``, then choose **Select Folder**.
224. In the Explorer pane, expand the **Python** folder to view the code files for this exercise.
225. Right-click on the `requirements.txt` file and select **Open in Integrated Terminal**.
226. In the terminal, enter the following command to install the required Python packages in a virtual environment:

```
python -m venv labenv

.\labenv\Scripts\Activate.ps1

pip install -r requirements.txt
```

227. Open the `.env` file, replace the `your_project_endpoint` placeholder with the endpoint for your project (copied from the project deployment resource in the Foundry Toolkit VS Code extension) and ensure that the `MODEL_DEPLOYMENT_NAME` variable is set to your model deployment name. Use **Ctrl+S** to save the file after making these changes.

Now you're ready to create an AI agent that uses MCP server tools to access external data sources and APIs.

Create a function for the agent to use

228. Open the `functions.py` file and review the existing code.

This file includes several functions that you can use as tools for your agent. The functions use sample files located in the data folder to retrieve information about astronomical events and locations.

229. Find the comment **Determine the next visible astronomical event for a given location** and add the following code:

```
python

# Determine the next visible astronomical event for a given location

def next_visible_event(location: str) -> str:

    """Returns the next visible astronomical event for a location."""

    today = int(datetime.now().strftime("%m%d"))

    loc = location.lower().replace(" ", "_")

    # Retrieve the next event visible from the location, starting with events later this year

    for name, event_type, date, date_str, locs in EVENTS:

        if loc in locs and date >= today:
```

```

        return json.dumps({"event": name, "type": event_type, "date": date_str, "visible_from":
sorted(locs)})

    return json.dumps({"message": f"No upcoming events found for {location}."})

```

This function checks the sample events data to find the next astronomical event that is visible from a specified location, and returns the event details as a JSON string. Next, let's create an agent that can use this function.

Connect to the Foundry project

230. Open the **agent.py** file.

> Tip: As you add code, be sure to maintain the correct indentation. Use the comment indentation levels as a guide.

231. Find the comment **Add references** and add the following code to import the classes you'll need to build an Azure AI agent that uses a function tool:

```

python

# Add references

from azure.ai.projects import AIProjectClient

from azure.identity import DefaultAzureCredential

from azure.ai.projects.models import PromptAgentDefinition, FunctionTool

from openai.types.responses.response_input_param import FunctionCallOutput,
ResponseInputParam

from functions import next_visible_event, calculate_observation_cost,
generate_observation_report

```

Notice that the functions you defined in the **functions.py** file are imported so they can be used as tools for the agent.

232. Find the comment **Connect to the project client** and add the following code:

```

python

# Connect to the project client

with (

    DefaultAzureCredential() as credential,

    AIProjectClient(endpoint=project_endpoint, credential=credential) as project_client,

    project_client.get_openai_client() as openai_client,

):

```

Define the function tools

In this task, you'll define each of the function tools that the agent can use. The parameters for each function tool are defined using a JSON schema, which specifies the name, type, description, and other attributes for each parameter of the function.

233. Find the comment ****Define the event function tool**** and add the following code:

```
python

# Define the event function tool

event_tool = FunctionTool(

    name="next_visible_event",

    description="Get the next visible event in a given location.",

    parameters={

        "type": "object",

        "properties": {

            "location": {

                "type": "string",

                "description": "continent to find the next visible event in (e.g. 'north_america', 'south_america', 'australia')",

            },

        },

        "required": ["location"],

        "additionalProperties": False,

    },

    strict=True,

)
```

234. Find the comment ****Define the observation cost function tool**** and add the following code:

```
python

# Define the observation cost function tool

cost_tool = FunctionTool(

    name="calculate_observation_cost",

    description="Calculate the cost of an observation based on the telescope tier, number of hours, and priority level.",

)
```

```

parameters={
    "type": "object",
    "properties": {
        "telescope_tier": {
            "type": "string",
            "description": "the tier of the telescope (e.g. 'standard', 'advanced', 'premium')",
        },
        "hours": {
            "type": "number",
            "description": "the number of hours for the observation",
        },
        "priority": {
            "type": "string",
            "description": "the priority level of the observation (e.g. 'low', 'normal', 'high')",
        },
    },
    "required": ["telescope_tier", "hours", "priority"],
    "additionalProperties": False,
},
strict=True,
)

```

235. Find the comment ****Define the observation report generation function tool**** and add the following code:

```

python

# Define the observation report generation function tool

report_tool = FunctionTool(
    name="generate_observation_report",
    description="Generate a report summarizing an astronomical observation",
    parameters={
        "type": "object",

```

```

"properties": {
  "event_name": {
    "type": "string",
    "description": "the name of the astronomical event being observed",
  },
  "location": {
    "type": "string",
    "description": "the location of the observer",
  },
  "telescope_tier": {
    "type": "string",
    "description": "the tier of the telescope used for the observation (e.g. 'standard',
'advanced', 'premium')",
  },
  "hours": {
    "type": "number",
    "description": "the number of hours the telescope was used for the observation",
  },
  "priority": {
    "type": "string",
    "description": "the priority level of the observation (e.g. 'low', 'normal', 'high')",
  },
  "observer_name": {
    "type": "string",
    "description": "the name of the person who conducted the observation",
  },
},
"required": ["event_name", "location", "telescope_tier", "hours", "priority", "observer_name"],
"additionalProperties": False,
},
strict=True,

```

)

Create the agent that uses the function tools

Now that you've defined the function tools, you can create an agent that can use those tools to complete tasks.

236. Find the comment ****Create a new agent with the function tools**** and add the following code:

```
python

# Create a new agent with the function tools

agent = project_client.agents.create_version(

    agent_name="astronomy-agent",

    definition=PromptAgentDefinition(

        model=model_deployment,

        instructions=

            """You are an astronomy observations assistant that helps users find

            information about astronomical events and calculate telescope rental costs.

            Use the available tools to assist users with their inquiries."""

        tools=[event_tool, cost_tool, report_tool],

    ),

)
```

Send a message to the agent and process the response

Now that you've created the agent with the function tools, you can send messages to the agent and process its responses.

237. Find the comment ****Create a thread for the chat session**** and add the following code:

```
python

# Create a thread for the chat session

conversation = openai_client.conversations.create()
```

This code creates the chat session with the agent.

238. Find the comment ****Create a list to hold function call outputs that will be sent back as input to the agent**** and add the following code:

```
python
```

```
# Create a list to hold function call outputs that will be sent back as input to the agent
```

```
input_list: ResponseInputParam = []
```

This list is created inside the chat loop so that each turn starts with a fresh set of function call outputs.

239. Find the comment ****Send a prompt to the agent**** and add the following code:

```
python
```

```
# Send a prompt to the agent
```

```
openai_client.conversations.items.create(  
    conversation_id=conversation.id,  
    items=[{"type": "message", "role": "user", "content": user_input}],  
)
```

240. Find the comment ****Retrieve the agent's response, which may include function calls**** and add the following code:

```
python
```

```
# Retrieve the agent's response, which may include function calls
```

```
response = openai_client.responses.create(  
    conversation=conversation.id,  
    extra_body={"agent_reference": {"name": agent.name, "type": "agent_reference"}},  
    input=input_list,  
)
```

```
# Check the run status for failures
```

```
if response.status == "failed":  
    print(f"Response failed: {response.error}")
```

In this code, you send a user prompt to the agent and retrieve the response. You also check if the response indicates a failure and print the error if so.

Process function calls and display the agent's response

241. Find the comment ****Process function calls**** and add the following code to handle any function calls made by the agent:

```
python
```

```

# Process function calls

for item in response.output:

    if item.type == "function_call":

        # Retrieve the matching function tool

        function_name = item.name

        result = None

        if item.name == "next_visible_event":

            result = next_visible_event(json.loads(item.arguments))

        elif item.name == "calculate_observation_cost":

            result = calculate_observation_cost(json.loads(item.arguments))

        elif item.name == "generate_observation_report":

            result = generate_observation_report(json.loads(item.arguments))

        # Append the output text

        input_list.append(

            FunctionCallOutput(

                type="function_call_output",

                call_id=item.call_id,

                output=result,

            )

        )

```

This code iterates through the items in the agent's response to check for any function calls. If a function call is found, it retrieves the corresponding function tool, executes the function with the provided arguments, and appends the result to the input list that will be sent back to the agent.

242. Find the comment ****Send function call outputs back to the model and retrieve a response**** and add the following code:

```

python

# Send function call outputs back to the model and retrieve a response

if input_list:

    response = openai_client.responses.create(

        conversation=conversation.id,

```

```

        input=input_list,

        extra_body={"agent_reference": {"name": agent.name, "type": "agent_reference"}},

    )

# Display the agent's response

print(f"AGENT: {response.output_text}")

```

This code checks if there are any function call outputs in the input list, and if so, it sends them back to the agent as input to retrieve an updated response. Finally, it prints the agent's response.

Note that the outputs are attached to the same conversation, so the function calls are resolved in conversation state and the agent's answer is saved to the chat history. Sending them back with `previous_response_id` instead would make the next message fail with "No tool output found for function call".

243. Find the comment ****Delete the agent when done**** and add the following code:

```

python

# Delete the agent when done

project_client.agents.delete_version(agent_name=agent.name, agent_version=agent.version)

print("Deleted agent.")

```

244. Review the complete code you've added to the file. It should now include sections that:

- Import necessary libraries
- Connect to the Foundry project and OpenAI client
- Define function tools for the agent to use
- Create an agent with those function tools
- Send a message to the agent and retrieve the response
- Process any function calls made by the agent and send the outputs back to the agent
- Display the agent's response
- Delete the agent when done

245. Save the code file (*CTRL+S*) when you have finished.

Run the agent application

246. In the integrated terminal, enter the following command to run the application:

```
az login
```

python agent.py

247. When prompted, enter a prompt such as:

Find me the next event I can see from South America and give me the cost for 5 hours of premium telescope time at normal priority.

Notice that this prompt asks the agent to use both of the function tools you defined: `next_visible_event` and `calculate_observation_cost`. The agent is able to invoke both functions in the same conversation turn, and use the outputs from those function calls to provide a helpful response to the user.

> Tip: If the app fails because the rate limit is exceeded. Wait a few seconds and try again. If there is insufficient quota available in your subscription, the model may not be able to respond.

You should see some output similar to the following:

output

AGENT: The next astronomical event you can observe from South America is the Jupiter-Venus Conjunction, taking place on May 1st.

The cost for 5 hours of premium telescope time at normal priority for this observation will be \$1,875.

248. Enter a follow-up prompt to generate an observation report, such as:

Generate that information in a report for Bellows College.

You should see a response similar to the following:

output

AGENT: Here is your report for Bellows College:

- Next visible astronomical event: Jupiter-Venus Conjunction
- Date: May 1st
- Visible from: South America
- Observation details:

- Telescope tier: Premium
- Duration: 5 hours
- Priority: Normal
- Observation cost: \$1,875

A formal report has been generated for Bellows College.

In the file explorer, you can see that a new file named `report-<event-type>.txt` has been created, which contains the generated report. You can open this file to view the contents of the report.

249. Enter ``quit`` to exit the application.

You can also use `deactivate` to exit the Python virtual environment in the terminal.

Clean up

When you've finished exploring the Foundry Toolkit for VS Code extension, you should clean up the resources to avoid incurring unnecessary Azure costs.

Delete your model

250. In VS Code, refresh the **Azure Resources** view.
251. Expand the **Models** subsection.
252. Right-click on your deployed model and select **Delete**.

Delete the resource group

253. Open the [Azure portal](https://portal.azure.com).
254. Navigate to the resource group containing your Microsoft Foundry resources.
255. Select **Delete resource group** and confirm the deletion.

Acceptance Evidence

Complete the pinned Microsoft Learning exercise `official/mslearn-ai-agents/Instructions/Exercises/02-agent-custom-tools.md`; capture its expected output, relevant version/endpoint, one measured result, and an explained failure or limitation.

Troubleshooting Checklist

256. Capture the full error and request/operation ID before retrying.
257. Confirm the endpoint, API/model version, region, identity, and RBAC scope.
258. Validate payload/schema fields and input content type.
259. Check quota, rate limits, network resolution, private connectivity, and service status.
260. Record the corrective action and rerun the acceptance check.

Lab 06 - Develop a vision-enabled chat app

Source folder: `labs/lab-06-develop-a-vision-enabled-chat-app`

Objectives

- Complete the exact Microsoft Learning exercise for the Computer vision domain.
- Capture a working result and explain its product-development implications.

Source

The complete, unchanged Microsoft exercise and assets are in [official/mslearn-ai-vision/Instructions/Exercises/01-gen-ai-vision.md](../official/mslearn-ai-vision/Instructions/Exercises/01-gen-ai-vision.md). The source revision and SHA-256 hashes are recorded in [OFFICIAL-SOURCE-MANIFEST.json](../OFFICIAL-SOURCE-MANIFEST.json). Read the source file for every numbered instruction; this course wrapper does not alter those steps.

Steps

Prepare

Read the source exercise prerequisites and use a trainer-provided Azure subscription or approved lab environment. Do not commit your own .env values.

Complete the official exercise

Follow the numbered steps in [01-gen-ai-vision.md](../official/mslearn-ai-vision/Instructions/Exercises/01-gen-ai-vision.md) exactly. Use its companion Labfiles or labfiles directory in the same source snapshot.

Capture evidence

Record the deployment or resource names, model and API versions, input, output, latency or quality measure, and one failure or limitation. Remove personal data and live keys.

Validation

The official exercise completes with its expected output; a second learner can reproduce the result from the source instructions and the recorded environment.

Timing

Approximately 30 minutes for the official exercise; trainer discussion and WSQ evidence review are scheduled separately.

Unchanged Microsoft Learning exercise instructions

Exact Markdown source and companion assets:

labs/official/mslearn-ai-vision/Instructions/Exercises/01-gen-ai-vision.md. Follow the numbered instructions below with the source assets. Microsoft Learning MIT license and pinned revision are in labs/official/ and the source manifest.

In this exercise, you use a generative AI model to generate responses to prompts that include images. You'll develop an app that provides AI assistance with fresh produce in a grocery store by using Microsoft Foundry and the OpenAI SDK.

While this exercise is based on the OpenAI Python SDK, you can develop AI chat applications using multiple language-specific SDKs; including:

- [OpenAI Projects for Microsoft .NET](https://www.nuget.org/packages/OpenAI)
- [OpenAI Projects for JavaScript](https://www.npmjs.com/package/openai)

This exercise takes approximately 30 minutes.

> Note: Some of the technologies used in this exercise are in preview or in active development. You may experience some unexpected behavior, warnings, or errors.

Prerequisites

Before starting this exercise, ensure you have:

- An active [Azure subscription](https://azure.microsoft.com/pricing/purchase-options/azure-account)
- [Visual Studio Code](https://code.visualstudio.com/) installed
- [Python version **3.13.xx**](https://www.python.org/downloads/release/python-31312/) installed*
- [Git](https://git-scm.com/install/) installed and configured
- [Azure CLI](https://learn.microsoft.com/cli/azure/install-azure-cli?view=azure-cli-latest) installed

> \ Python 3.14 is available, but some dependencies are not yet compiled for that release. The lab has been successfully tested with Python 3.13.12.

Create a Microsoft Foundry project

Microsoft Foundry uses projects to organize models, resources, data, and other assets used to develop an AI solution.

261. In a web browser, open the [Microsoft Foundry portal](https://ai.azure.com) at `https://ai.azure.com` to start building; signing in using your Azure credentials. Close any tips or quick start panes that are opened the first time you sign in.
262. If it is not already enabled, in the tool bar the top of the page, enable the **New Foundry** option. Then, if prompted, create a new project with a unique name; expanding the **Advanced options** area to specify the following settings for your project:

Foundry resource: Use the default name for your resource (usually {project_name}-resource)

Subscription: Your Azure subscription

Resource group: Create or select a resource group

Region: Select any available region

263. Wait for your project to be created. Then, on the home page for your project, note that the API key, project endpoint, and Azure OpenAI endpoint are displayed here.

> TIP: You're going to need the Azure OpenAI endpoint later!

Deploy a model

You'll need a model that can process image-based input.

264. Now you're ready to explore models. On the **Discover** page, select the **Models** tab to view the Microsoft Foundry model catalog.
265. Search for and deploy the `gpt-5.2` model using the default settings. Deployment may take a minute or so.

> Tip: Model deployments are subject to regional quotas. If you don't have enough quota to deploy the model in your project's region, you can use a different model - such as gpt-5.2-mini, or gpt-4o. Alternatively, you can create a new project in a different region.

266. When the model has been deployed, view the model playground page that is opened, in which you can chat with the model.

> TIP: Note the model deployment name (which by default should be gpt-5.2) - you'll need this later!

Test the model in the playground

Now you can test your model deployment with an image-based prompt in the chat playground.

267. In a new browser tab, download [mango.jpeg](https://microsoftlearning.github.io/mslearn-ai-vision/Labfiles/gen-ai-vision/mango.jpeg) from `https://microsoftlearning.github.io/mslearn-ai-vision/Labfiles/gen-ai-vision/mango.jpeg` and save it to a folder on your local file system.
268. Navigate back to the chat playground page for your model deployment in the Foundry portal.
269. In the main chat session panel, under the chat input box, use the attach button (**📎**) to upload the *mango.jpeg* image file, and then add the text `What desserts could I make with this fruit?` and submit the prompt.

![Screenshot of the chat playground page.](../media/chat-playground-image-new.png)

270. Review the response, which should hopefully provide relevant guidance for desserts you can make using a mango.

Create a client application

Now that you've deployed the model, you can use the deployment in a client application.

Get application files from GitHub

The initial application files you'll need to develop the translation application are provided in a GitHub repo.

271. Open Visual Studio Code.
272. Open the command palette (*Ctrl+Shift+P*) and use the `Git:clone` command to clone the `https://github.com/microsoftlearning/mslearn-ai-vision` repo to a local folder (it doesn't matter which one). Then open it.

You may be prompted to confirm you trust the authors.

273. In Visual Studio Code, view the **Extensions** pane; and if it is not already installed, install the **Python** extension.
274. In the **Command Palette**, use the command `python:select interpreter`. Then select an existing environment if you have one, or create a new **Venv** environment based on your Python 3.13.x installation.

> Tip: If you are prompted to install dependencies, you can install the ones in the requirements.txt file in the labfiles/gen-ai-vision/python folder; but it's OK if you don't - we'll install them later!

Prepare the application configuration

275. After the repo has been cloned, open the folder in VS Code (**File > Open Folder**), and navigate to the `Labfiles/gen-ai-vision/python` folder.
276. In the VS Code Explorer pane, review the files in the folder:

.env - A configuration file for application settings.

image-chat-app.py - The Python code file for the image application.

requirements.txt - A file listing the package dependencies.

mystery-fruit.jpeg - An image of a fruit.

277. In the **Explorer** pane, in the **python** folder, select the **.env** file to open it. Then update the configuration values to include the **Azure OpenAI endpoint** for your Foundry resource, and the model deployment name for the generative AI model you deployed.

> Important: Be sure to add the `https://{foundry-resource-name}.openai.azure.com/openai/v1/` Azure OpenAI endpoint, <u>not</u> the project endpoint!

Save the modified configuration file.

278. In the **Explorer** pane, right-click the **python** folder containing the application files, and select **Open in integrated terminal** (or open a terminal in the **Terminal** menu and navigate to the `*labfiles/gen-ai-vision/python*` folder.)

> Note: Opening the terminal in Visual Studio Code will automatically activate the Python environment. You may need to enable running scripts on your system.

279. Ensure that the terminal is open in the **Labfiles/gen-ai-vision/python** folder with the prefix **`*(.venv)*`** to indicate that the Python environment you created is active.

280. Install the required Python packages by running the following command:

```
pip install -r requirements.txt
```

Write code to get an OpenAI chat client for your model

> Tip: As you add code, be sure to maintain the correct indentation.

281. In VS Code, open the `image-chat-app.py` file.

282. In the code file, note the existing statements that have been added at the top of the file to import the necessary SDK namespaces. Then, Find the comment **`**Add references**`**, add the following code to reference the namespaces in the libraries you installed previously:

```
python
```

```
# Add references
```

```
from openai import OpenAI
```

```
from azure.identity import DefaultAzureCredential, get_bearer_token_provider
```

283. In the **`**main**`** function, under the comment **`**Get configuration settings**`**, note that the code loads the project connection string and model deployment name values you defined in the configuration file.

284. Find the comment **`**Create an OpenAI client**`**, and add the following code to connect to your Azure AI Foundry project:

> Tip: Be careful to maintain the correct indentation level for your code.

```
python
```

```
# Create an OpenAI client
```

```
credential = DefaultAzureCredential()
```

```
token_provider = get_bearer_token_provider(credential, "https://ai.azure.com/.default")
```

```
client = OpenAI(
```

```
    base_url=openai_endpoint,
```

```
api_key=token_provider()

)
```

Write code to submit a URL-based image prompt

285. Note that the code includes a loop to allow a user to input a prompt until they enter "quit". Then in the loop section, find the comment ****Get a response to image input****, add the following code to submit a prompt that includes the following image:

```
![A photo of an orange.](../media/orange.jpeg)

python

# Get a response to image input

image_url = "https://microsoftlearning.github.io/mslearn-ai-vision/Labfiles/gen-ai-vision/
orange.jpeg"

response = client.responses.create(

    model=model_deployment,

    input=[

        {"role": "developer", "content": system_message},

        { "role": "user", "content": [

            { "type": "input_text", "text": prompt},

            { "type": "input_image", "image_url": image_url}

        ]}

    ]

)

print(response.output_text)
```

286. Save your changes to the code file.

Sign into Azure and run the app

287. In the terminal pane, use the following command to sign into Azure.

```
powershell

az login
```

> Note: In most scenarios, just using az login will be sufficient. However, if you have subscriptions in multiple tenants, you may need to specify the tenant by using the --tenant parameter. See [Sign

into Azure interactively using the Azure CLI](<https://learn.microsoft.com/cli/azure/authenticate-azure-cli-interactively>) for details.

288. When prompted, follow the instructions to sign into Azure. Then complete the sign in process in the command line, viewing (and confirming if necessary) the details of the subscription containing your Foundry resource.

289. After you have signed in, enter the following command to run the application:

```
python image-chat-app.py
```

290. When prompted, enter the following prompt:

```
Suggest some recipes that include this fruit
```

291. Review the response. Then enter `quit` to exit the program.

Modify the code to upload a local image file

292. In the code editor for your app code, in the loop section, find the code you added previously under the comment `**Get a response to image input**`. Then modify the code as follows, to upload this local image file:

```
![A photo of a dragon fruit.](../media/mystery-fruit.jpeg)

python

# Get a response to image input

image_path = Path("mystery-fruit.jpeg")

image_format = "jpeg"

with open(image_path, "rb") as image_file:

    image_data = base64.b64encode(image_file.read()).decode("utf-8")

data_url = f"data:image/{image_format};base64,{image_data}"

response = client.responses.create(

    model=model_deployment,

    input=[

        {"role": "developer", "content": system_message},

        { "role": "user", "content": [

            { "type": "input_text", "text": prompt},

            { "type": "input_image", "image_url": data_url}

        ]}

    ]

)
```

```

    }}
]
)

print(response.output_text)

```

293. Use the ****CTRL+S**** command to save your changes to the code file.

294. In the terminal, enter the following command to run the app:

```
python image-chat-app.py
```

295. When prompted, enter the following prompt:

What is this fruit? What recipes could I use it in?

296. Review the response. Then enter ``quit`` to exit the program.

> Note: In this simple app, we haven't implemented logic to retain conversation history; so the model will treat each prompt as a new request with no context of the previous prompt.

Clean up

If you've finished exploring Azure AI Foundry portal, you should delete the resources you have created in this exercise to avoid incurring unnecessary Azure costs.

297. Open the [Azure portal](https://portal.azure.com) and view the contents of the resource group where you deployed the resources used in this exercise.

298. On the toolbar, select ****Delete resource group****.

299. Enter the resource group name and confirm that you want to delete it.

Acceptance Evidence

Complete the pinned Microsoft Learning exercise [official/mslearn-ai-vision/Instructions/Exercises/01-gen-ai-vision.md](https://www.microsoft.com/learning/exercise/01-gen-ai-vision); capture its expected output, relevant version/endpoint, one measured result, and an explained failure or limitation.

Troubleshooting Checklist

300. Capture the full error and request/operation ID before retrying.

301. Confirm the endpoint, API/model version, region, identity, and RBAC scope.

302. Validate payload/schema fields and input content type.

303. Check quota, rate limits, network resolution, private connectivity, and service status.

304. Record the corrective action and rerun the acceptance check.

Lab 07 - Analyze text

Source folder: labs/lab-07-analyze-text

Objectives

- Complete the exact MicrosoftLearning exercise for the Text analysis domain.
- Capture a working result and explain its product-development implications.

Source

The complete, unchanged Microsoft exercise and assets are in [official/mslearn-ai-language/Instructions/Exercises/01-analyze-text.md](../official/mslearn-ai-language/Instructions/Exercises/01-analyze-text.md). The source revision and SHA-256 hashes are recorded in [OFFICIAL-SOURCE-MANIFEST.json](../OFFICIAL-SOURCE-MANIFEST.json). Read the source file for every numbered instruction; this course wrapper does not alter those steps.

Steps

Prepare

Read the source exercise prerequisites and use a trainer-provided Azure subscription or approved lab environment. Do not commit your own .env values.

Complete the official exercise

Follow the numbered steps in [01-analyze-text.md](../official/mslearn-ai-language/Instructions/Exercises/01-analyze-text.md) exactly. Use its companion Labfiles or labfiles directory in the same source snapshot.

Capture evidence

Record the deployment or resource names, model and API versions, input, output, latency or quality measure, and one failure or limitation. Remove personal data and live keys.

Validation

The official exercise completes with its expected output; a second learner can reproduce the result from the source instructions and the recorded environment.

Timing

Approximately 30 minutes for the official exercise; trainer discussion and WSQ evidence review are scheduled separately.

Unchanged MicrosoftLearning exercise instructions

Exact Markdown source and companion assets:

labs/official/mslearn-ai-language/Instructions/Exercises/01-analyze-text.md. Follow the numbered instructions below with the source assets. MicrosoftLearning MIT license and pinned revision are in labs/official/ and the source manifest.

Azure Language in Foundry Tools supports analysis of text, including language detection, entity recognition, and PII redaction.

For example, suppose a travel agency wants to process hotel reviews that have been submitted to the company's web site. By using the Azure Language, they can determine the language each review is written in, identify named entities, such as places, landmarks, or people mentioned in the reviews, and redact any personally identifiable information before publishing them on the company's

website. In this exercise, you'll use the Azure Language Python SDK for text analytics to implement a simple hotel review application.

While this exercise is based on Python, you can develop text analytics applications using multiple language-specific SDKs; including:

The code used in this exercise is based on the for Microsoft Foundry Tools SDK for Python. You can develop similar solutions using the SDKs for Microsoft .NET, JavaScript, and Java. Refer to [Microsoft Foundry SDK client libraries](https://learn.microsoft.com/azure/ai-foundry/how-to/develop/sdk-overview) for details.

This exercise takes approximately 30 minutes.

> Note: Some of the technologies used in this exercise are in preview or in active development. You may experience some unexpected behavior, warnings, or errors.

Prerequisites

Before starting this exercise, ensure you have:

- An active [Azure subscription](https://azure.microsoft.com/pricing/purchase-options/azure-account)
- [Visual Studio Code](https://code.visualstudio.com/) installed
- [Python version **3.13.xx**](https://www.python.org/downloads/release/python-31312/) installed*
- [Git](https://git-scm.com/install/) installed and configured
- [Azure CLI](https://learn.microsoft.com/cli/azure/install-azure-cli?view=azure-cli-latest) installed

> \ Python 3.14 is available, but some dependencies are not yet compiled for that release. The lab has been successfully tested with Python 3.13.12.

Create a Microsoft Foundry project

Microsoft Foundry uses projects to organize models, resources, data, and other assets used to develop an AI solution.

305. In a web browser, open the [Microsoft Foundry portal](https://ai.azure.com) at `https://ai.azure.com` and sign in using your Azure credentials. Close any tips or quick start panes that are opened the first time you sign in, and if necessary use the Foundry logo at the top left to navigate to the home page.

306. If it is not already enabled, in the tool bar the top of the page, enable the **New Foundry** option. Then, if prompted, create a new project with a unique name; expanding the **Advanced options** area to specify the following settings for your project:

- Foundry resource: Use the default name for your resource (usually {project_name}-resource)
- Subscription: Your Azure subscription
- Resource group: Create or select a resource group
- Region: Select any available region

> Note: Use a recommended Microsoft Foundry region. Model availability may vary by region.

307. Select **Create**. Wait for your project to be created.

308. On the home page for your project, note that the API key, project endpoint, and OpenAI endpoint are displayed here.

> TIP: You're going to need the project endpoint later!

Get the application files from GitHub

The initial application files you'll need to develop the review analysis application are provided in a GitHub repo.

309. Open Visual Studio Code.

310. Open the command palette (*Ctrl+Shift+P*) and use the `Git:clone` command to clone the `https://github.com/microsoftlearning/mslearn-ai-language` repo to a local folder (it doesn't matter which one). Then open it.

You may be prompted to confirm you trust the authors.

311. After the repo has been cloned, in the Explorer pane, navigate to the folder containing the application code files at `**/Labfiles/01-analyze-text/Python/text-analysis**`. The application files include:

- reviews (a subfolder containing the review documents)
- .env (the application configuration file)
- requirements.txt (the Python package dependencies that need to be installed)
- text-analysis.py (the code file for the application)

Configure your application

312. In Visual Studio Code, view the `**Extensions**` pane; and if it is not already installed, install the `**Python**` extension.

313. In the `**Command Palette**`, use the command `python:select interpreter`. Then select an existing environment if you have one, or create a new `**Venv**` environment based on your Python 3.13.x installation.

> Tip: If you are prompted to install dependencies, you can install the ones in the requirements.txt file in the /Labfiles/01-analyze-text/Python/text-analysis folder; but it's OK if you don't - we'll install them later!

> Tip: If you prefer to use the terminal, you can create your Venv environment with `python -m venv labenv`, then activate it with `\labenv\Scripts\activate`.

314. In the `**Explorer**` pane, right-click the `**text-analysis**` folder containing the application files, and select `**Open in integrated terminal**` (or open a terminal in the `**Terminal**` menu and navigate to the `*/Labfiles/01-analyze-text/Python/text-analysis*` folder.)

> Note: Opening the terminal in Visual Studio Code will automatically activate the Python environment. You may need to enable running scripts on your system.

315. Ensure that the terminal is open in the `**text-analysis**` folder with the prefix `**(.venv)**` to indicate that the Python environment you created is active.

316. Install the Azure Language Text Analytics SDK and other required packages by running the following command:

```
pip install -r requirements.txt
```

317. In the **Explorer** pane, in the **text-analysis** folder, select the **.env** file to open it. Then update the configuration values to include the **endpoint** (up to the **.com** domain) for your Foundry project (copy these from the Foundry portal).

> Important: Modify the pasted endpoint to remove the `"/api/projects/{project_name}"` suffix - the endpoint should be `https://{your-foundry-resource-name}.services.ai.azure.com`.

Save the modified configuration file.

Add code to connect to your Azure AI Language resource

318. In the **Explorer** pane, in the **text-analysis** folder, open the **text-analysis.py** file.

319. Review the existing code. You will add code to work with the Azure Language Text Analytics SDK.

> Tip: As you add code to the code file, be sure to maintain the correct indentation.

320. At the top of the code file, under the existing namespace references, find the comment **Import namespaces** and add the following code to import the namespaces you will need to use the Text Analytics SDK:

```
python

# import namespaces

from azure.identity import DefaultAzureCredential

from azure.ai.textanalytics import TextAnalyticsClient
```

321. In the **main** function, note that code to load the endpoint from the configuration file has already been provided. Then find the comment **Create client using endpoint**, and add the following code to create a client for the Text Analysis API:

```
Python

# Create client using endpoint

credential = DefaultAzureCredential()

ai_client = TextAnalyticsClient(endpoint=foundation_endpoint, credential=credential)
```

322. Save the changes to the code file. Then, in the terminal pane, use the following command to sign into Azure.

```
powershell

az login
```

> Note: In most scenarios, just using `az login` will be sufficient. However, if you have subscriptions in multiple tenants, you may need to specify the tenant by using the `--tenant` parameter. See [Sign

into Azure interactively using the Azure CLI](<https://learn.microsoft.com/cli/azure/authenticate-azure-cli-interactively>) for details.

323. When prompted, follow the instructions to sign into Azure. Then complete the sign in process in the command line, viewing (and confirming if necessary) the details of the subscription containing your Foundry resource.

324. After you have signed in, enter the following command to run the application:

```
python text-analysis.py
```

325. Observe the output as the code should run without error, displaying the contents of each review text file in the **reviews** folder. The application successfully creates a client for the Text Analytics API but doesn't make use of it. We'll fix that in the next section.

Add code to detect language

Now that you have created a client for the API, let's use it to detect the language in which each review is written.

326. In the code editor, find the comment **Get language**. Then add the code necessary to detect the language in each review document:

```
python

# Get language

detectedLanguage = ai_client.detect_language(documents=[text])[0]

print("\nLanguage: {}".format(detectedLanguage.primary_language.name))
```

> Note: In this example, each review is analyzed individually, resulting in a separate call to the service for each file. An alternative approach is to create a collection of documents and pass them to the service in a single call. In both approaches, the response from the service consists of a collection of documents; which is why in the Python code above, the index of the first (and only) document in the response ([0]) is specified.

327. Save your changes. Then re-run the program.

328. Observe the output, noting that this time the language for each review is identified.

Add code to extract entities

Often, documents or other bodies of text mention people, places, time periods, or other entities. The text Analytics API can detect multiple categories (and subcategories) of entity in your text.

329. In the code editor, find the comment **Get entities**. Then, add the code necessary to identify entities that are mentioned in each review:

```
python

# Get entities

entities = ai_client.recognize_entities(documents=[text])[0].entities
```

```

if len(entities) > 0:

    print("\nEntities")

    for entity in entities:

        print('\t{} {}'.format(entity.text, entity.category))

```

330. Save your changes and re-run the program.
331. Observe the output, noting the entities that have been detected in the text.

Add code to redact PII

Often, privacy policies and legislation can require that personally identifiable information (PII), such as names, addresses, phone numbers, and other private details be redacted from documents.

332. In the code editor, find the comment ****Get PII****. Then, add the code necessary to identify PII entities that are mentioned in each review:

```

python

# Get PII

pii_result = ai_client.recognize_pii_entities(documents=[text])[0]

pii_entities = pii_result.entities

if len(pii_entities) > 0:

    print("\nPII Entities")

    for pii_entity in pii_entities:

        print('\t{} {}'.format(pii_entity.text, pii_entity.category))

    print("Redacted Text:\n {}".format(pii_result.redacted_text))

```

333. Save your changes and re-run the program.
334. Observe the output, noting the PII entities that are identified, and reviewing the redacted version of each document that is produced.

Clean up

If you've finished exploring Azure Language in Foundry Tools, you should delete the resources you have created in this exercise to avoid incurring unnecessary Azure costs.

335. Open the [Azure portal](https://portal.azure.com) and view the contents of the resource group where you deployed the resources used in this exercise.
336. On the toolbar, select ****Delete resource group****.
337. Enter the resource group name and confirm that you want to delete it.

Acceptance Evidence

Complete the pinned Microsoft Learning exercise [official/mslearn-ai-language/Instructions/Exercises/01-analyze-text.md](#); capture its expected output, relevant version/endpoint, one measured result, and an explained failure or limitation.

Troubleshooting Checklist

- 338. Capture the full error and request/operation ID before retrying.
- 339. Confirm the endpoint, API/model version, region, identity, and RBAC scope.
- 340. Validate payload/schema fields and input content type.
- 341. Check quota, rate limits, network resolution, private connectivity, and service status.
- 342. Record the corrective action and rerun the acceptance check.

Lab 08 - Develop a text analysis agent

Source folder: `labs/lab-08-develop-a-text-analysis-agent`

Objectives

- Complete the exact Microsoft Learning exercise for the Text analysis domain.
- Capture a working result and explain its product-development implications.

Source

The complete, unchanged Microsoft exercise and assets are in [\[official/mslearn-ai-language/Instructions/Exercises/02-language-agent.md\]\(../official/mslearn-ai-language/Instructions/Exercises/02-language-agent.md\)](#). The source revision and SHA-256 hashes are recorded in [\[OFFICIAL-SOURCE-MANIFEST.json\]\(../OFFICIAL-SOURCE-MANIFEST.json\)](#). Read the source file for every numbered instruction; this course wrapper does not alter those steps.

Steps

Prepare

Read the source exercise prerequisites and use a trainer-provided Azure subscription or approved lab environment. Do not commit your own `.env` values.

Complete the official exercise

Follow the numbered steps in [\[02-language-agent.md\]\(../official/mslearn-ai-language/Instructions/Exercises/02-language-agent.md\)](#) exactly. Use its companion `Labfiles` or `labfiles` directory in the same source snapshot.

Capture evidence

Record the deployment or resource names, model and API versions, input, output, latency or quality measure, and one failure or limitation. Remove personal data and live keys.

Validation

The official exercise completes with its expected output; a second learner can reproduce the result from the source instructions and the recorded environment.

Timing

Approximately 30 minutes for the official exercise; trainer discussion and WSQ evidence review are scheduled separately.

Unchanged Microsoft Learning exercise instructions

Exact Markdown source and companion assets:

labs/official/mslearn-ai-language/Instructions/Exercises/02-language-agent.md. Follow the numbered instructions below with the source assets. Microsoft Learning MIT license and pinned revision are in labs/official/ and the source manifest.

Azure Language in Foundry Tools supports analysis of text, including language detection, entity recognition, and PII redaction.

You can use the service directly in an application through its REST API and several language-specific SDKs. You can also use the Azure Language in Foundry Tools MCP server to integrate its capabilities into an AI agent, which is what you'll do in this exercise.

The code used in this exercise is based on the Microsoft Foundry SDK for Python. You can develop similar solutions using the SDKs for Microsoft .NET, JavaScript, and Java. Refer to [Microsoft Foundry SDK client libraries](https://learn.microsoft.com/azure/ai-foundry/how-to/develop/sdk-overview) for details.

This exercise takes approximately 30 minutes.

> Note: Some of the technologies used in this exercise are in preview or in active development. You may experience some unexpected behavior, warnings, or errors.

Prerequisites

Before starting this exercise, ensure you have:

- An active [Azure subscription](https://azure.microsoft.com/pricing/purchase-options/azure-account)
- [Visual Studio Code](https://code.visualstudio.com/) installed
- [Python version **3.13.xx**](https://www.python.org/downloads/release/python-31312/) installed*
- [Git](https://git-scm.com/install/) installed and configured
- [Azure CLI](https://learn.microsoft.com/cli/azure/install-azure-cli?view=azure-cli-latest) installed

> \ Python 3.14 is available, but some dependencies are not yet compiled for that release. The lab has been successfully tested with Python 3.13.12.

Create a Microsoft Foundry project

Microsoft Foundry uses projects to organize models, resources, data, and other assets used to develop an AI solution.

343. In a web browser, open the [Microsoft Foundry portal](https://ai.azure.com) at `https://ai.azure.com` and sign in using your Azure credentials. Close any tips or quick start panes that are opened the first time you sign in, and if necessary use the Foundry logo at the top left to navigate to the home page.
344. If it is not already enabled, in the tool bar the top of the page, enable the **New Foundry** option. Then, if prompted, create a new project with a unique name; expanding the **Advanced options** area to specify the following settings for your project:
 - Foundry resource: Use the default name for your resource (usually {project_name}-resource)
 - Subscription: Your Azure subscription
 - Resource group: Create or select a resource group

- Region: Select any available region

> Note: Use a recommended Microsoft Foundry region. Model availability may vary by region.

> TIP: Remember (or make a note of) the Foundry resource name - you're going to need it later!

345. Select ****Create****. Wait for your project to be created.

346. On the home page for your project, note that the API key, project endpoint, and OpenAI endpoint are displayed here.

> TIP: Copy the project key to the clipboard - you're going to need it later!

Create an agent

Now that you have a Foundry project, you can create an agent.

347. Now you're ready to ****Start building****. Select ****Create agents**** (or on the ****Build**** page, select the ****Agents**** tab); and create a new agent named ``Text-Analysis-Agent``.

When ready, your agent opens in the agent playground.

348. In the model drop-down list, ensure that a ****gpt-5**** model has been deployed and selected for your agent.

> Note: Instant Inference quota may take time to initialize. If you see `deployment_disabled` with "Instant inference quota is still being initialized for this subscription", you can wait and retry, or deploy a model to continue this exercise. Select Models in the side panel, search for gpt-5, and select Custom deploy. Keep the default settings, deploy, and note the deployment name for later use. Once deployment completes, select it in your agent's model drop-down list, save, and retry your prompt.

349. Assign your agent the following ****Instructions****:

You are an AI agent that assists users by helping them analyze text.

350. Use the ****Save**** button to save the changes.

351. Test the agent by entering the following prompt in the ****Chat**** pane:

What can you help me with?

The agent should respond with an appropriate answer based on its instructions.

Create an Azure Language in Foundry Tools connection

Foundry includes an MCP server for Azure Language in Foundry Tools, which you can connect to your project and use in your agent.

352. In the navigation pane on the left, select the ****Tools**** page.

353. Within the Tools page select the ****Tools**** tab.

354. Connect a tool; selecting ****Azure Language in Foundry Tools**** in the ****Catalog**** and connecting it to an endpoint. specifying the following configuration

- Name: A unique name for your tool/

- Remote MCP Server endpoint:

<https://{foundry-resource-name}.cognitiveservices.azure.com/language/mcp?api-version=2025-11-15-preview>

- Parameters: foundry-resource-name: Your foundry resource name

- Authentication: Key-based:

- Ocp-Apim-Subscription-Key: API Key for your Foundry project

> Note: If key-based authentication is disabled by a policy in your Azure subscription, you can use Entra ID authentication to connect the agent to the Azure Language service.

355. Wait for the MCP tool connection to be created, and then view its details page.

356. On the details page for the Azure Language in Foundry Tools connection, select **Use in an agent**, and then select the **Text-Analysis-Agent** agent you created previously.

The agent should open in the playground, with the Azure Language in Foundry Tools tool connected.

> Note: In some cases, the MCP tool is not added to the agent automatically. Verify that Azure Language in Foundry Tools appears in the agent's tool list. If it does not, add it manually from the available tools for the agent.

Test the Azure Language tool in the playground

Now let's test the agent's ability to use the tool you connected.

357. In the agent playground for the **Text-Analysis-Agent** agent, modify the instructions as follows:

You are an AI agent that assists users by helping them analyze text. Use the Azure Language tool to perform text analysis tasks.

358. Use the **Save** button to save the changes.

359. Test the agent by entering the following prompt in the **Chat** pane:

Identify the PII entities in this article, and generate a redacted version:

Microsoft was founded on April 4, 1975, by childhood friends Bill Gates (then 19) and Paul Allen (22) after they were inspired by the Altair 8800, one of the first personal computers, featured on the cover of Popular Electronics. They contacted the Altair's maker, MITS, and successfully developed a version of the BASIC programming language, despite initially not owning the machine themselves. The pair formed a partnership called "Micro□Soft" in Albuquerque, New Mexico, close to MITS's headquarters, with the goal of writing software for emerging microcomputers.

In the late 1970s, Microsoft grew by supplying programming languages to multiple hardware vendors, then relocated to the Seattle area in 1979. A pivotal moment came in 1980 when Microsoft partnered with IBM to provide an operating system for the IBM PC, leading to MS□DOS and

establishing the company's dominance in personal computing. Gates guided the company's long-term strategy as CEO, while Allen contributed key technical vision in its early years, setting Microsoft on a path that would reshape the software industry.

360. When prompted, approve use of the Azure Language tool by selecting ****Always approve all Azure Language in Foundry Tools tools**** (you may need to do this twice because the prompt asked for two distinct text analysis tasks).

> Note: Depending on your environment, you may occasionally receive a 403, 502, or other transient error after approving the tool. These errors are usually not blocking. Refresh the page and retry the same prompt.

361. Review the response, which should identify any personally identifiable information in the article about the founding of Microsoft, and create a version of the article with this information redacted.

362. Review the ****Logs**** for the chat and verify that the Azure Language tool was used by the agent to process the prompt.

Configure tool approval

As you've seen in the playground, to use the tool, the agent needs approval.

363. In the playground, in the list of ****Tools**** under the ****Instructions****, in the menu for the Azure language tool you added, select ****Configure****.

364. Ensure that the ****Approval setting for tools in this MCP server for this agent**** setting is ****Always auto-approve all tools**** (if not, change it and add it).

365. Save any changes to the agent.

Create a client application

Now that you have a working agent, you can create a client application that uses it.

Get the application files from GitHub

366. Open Visual Studio Code.

367. Open the command palette (*Ctrl+Shift+P*) and use the ``Git:clone`` command to clone the ``https://github.com/microsoftlearning/mslearn-ai-language`` repo to a local folder (it doesn't matter which one). Then open it.

You may be prompted to confirm you trust the authors.

368. After the repo has been cloned, in the Explorer pane, navigate to the folder containing the application code files at *****/Labfiles/02-language-agent/Python/text-agent****. The application files include:

- .env (the application configuration file)
- requirements.txt (the Python package dependencies that need to be installed)
- text-agent.py (the code file for the application)

Configure the application

369. In Visual Studio Code, view the ****Extensions**** pane; and if it is not already installed, install the ****Python**** extension.

370. In the **Command Palette**, use the command ``python:select interpreter``. Then select an existing environment if you have one, or create a new **Venv** environment based on your Python 3.1x installation.

> Tip: If you are prompted to install dependencies, you can install the ones in the requirements.txt file in the /Labfiles/02-language-agent/Python/text-agent folder; but it's OK if you don't, we'll install them later.

> Tip: If you prefer to use the terminal, you can create your Venv environment with `python -m venv labenv`, then activate it with `\labenv\Scripts\activate`.

371. In the **Explorer** pane, right-click the **text-agent** folder containing the application files, and select **Open in integrated terminal** (or open a terminal in the **Terminal** menu and navigate to the */Labfiles/02-language-agent/Python/text-agent* folder.)

> Note: Opening the terminal in Visual Studio Code will automatically activate the Python environment. You may need to enable running scripts on your system.

372. Ensure that the terminal is open in the **text-agent** folder with the prefix **(.venv)** to indicate that the Python environment you created is active.

373. Install the Foundry SDK package, the Azure Identity package, and other required packages by running the following command:

```
pip install -r requirements.txt
```

374. In the **Explorer** pane, in the **text-agent** folder, select the **.env** file to open it. Then update the configuration values to include your project **endpoint** (from the project home page in Foundry Portal) and the name of your agent (which should be **Text-Analysis-Agent** - note that this name is case-sensitive).

375. Save the modified configuration file.

Implement application code

376. In the **Explorer** pane, in the **text-agent** folder, open the **text-agent.py** file.

377. Review the existing code. You will add code to submit prompts to your agent.

> Tip: As you add code to the code file, be sure to maintain the correct indentation.

378. At the top of the code file, under the existing namespace references, find the comment **Import namespaces** and add the following code to import the namespaces you will need:

```
python

# import namespaces

from azure.identity import DefaultAzureCredential

from azure.ai.projects import AIProjectClient
```

379. In the ****main**** function, note that code to load the endpoint from the configuration file has already been provided. Then find the comment ****Get project client****, and add the following code to create a client for your Foundry project:

```
python

# Get project client

project_client = AIProjectClient(

    endpoint=foundry_endpoint,

    credential=DefaultAzureCredential(),

)
```

380. Find the comment ****Get an OpenAI client****, and add the following code to get an OpenAI client with which to call your agent.

```
python

# Get an OpenAI client

openai_client = project_client.get_openai_client()
```

381. Find the comment ****Use the agent to get a response****, and add the following code to submit a user prompt to your agent, and display the response.

```
python

# Use the agent to get a response

prompt = input("User prompt: ")

response = openai_client.responses.create(

    input=[{"role": "user", "content": prompt}],

    extra_body={"agent_reference": {"name": agent_name, "type": "agent_reference"}},

)

print(f"{agent_name}: {response.output_text}")
```

382. Save the changes you made to the code file.

Test the client application

Now let's test the application by running it in a Python environment and authenticating the connection to your project.

383. In the Visual Studio Code terminal, enter the following command to sign into Azure

powershell

az login

> Note: In most scenarios, just using az login will be sufficient. However, if you have subscriptions in multiple tenants, you may need to specify the tenant by using the --tenant parameter. See [Sign into Azure interactively using the Azure CLI](https://learn.microsoft.com/cli/azure/authenticate-azure-cli-interactively) for details.

384. When prompted, follow the instructions to sign into Azure. Then complete the sign in process in the command line, viewing (and confirming if necessary) the details of the subscription containing your Foundry resource.

385. After you have signed in, enter the following command to run the application:

powershell

python text-agent.py

386. When prompted, enter the following prompt:

Extract named entities from the following text: "Pierre and I went to Paris on July 14th."

387. Review the response, which should identify named people, places, and dates.

View tool details

The Azure Language in Foundry Tools tool provides a wide range of functionality, and the agent must select the appropriate function to call. We can see the options available in the agent's response.

388. In the `text-agent.py` code file, add the following line immediately after the `*print(f"{agent_name}: {response.output_text}")*` line you added previously (before the `*except Exception as ex:*` line):

python

`print(f"\nResponse Details: {response.model_dump_json(indent=2)}")`

389. Save the changes to the code file.

390. In the terminal, re-enter the command to run the application (`python text-agent.py`).

391. When prompted, enter the following command:

Tell me what entities and dates are mentioned in this review, and whether it is positive or negative:
"I booked my flight to Paris in July with Margie's Travel, and it was fantastic!"

392. Review the response (you may need to scroll quite far up to see it), which should identify entities and dates, and determine the sentiment of the text.
393. Review the JSON response details, which indicate each of the tools available to the agent. In this case, it should have used the `**extract_named_entities_from_text**` and `**detect_sentiment_from_text**` tools within Azure Language in Foundry Tools.

Clean up resources

If you're finished exploring the Azure Language service, you can delete the resources you created in this exercise. Here's how:

394. In the Azure portal, browse to the Foundry resource you created in this lab.
395. On the resource page, select `**Delete**` and follow the instructions to delete the resource.

Acceptance Evidence

Complete the pinned Microsoft Learning exercise `official/mslearn-ai-language/Instructions/Exercises/02-language-agent.md`; capture its expected output, relevant version/endpoint, one measured result, and an explained failure or limitation.

Troubleshooting Checklist

396. Capture the full error and request/operation ID before retrying.
397. Confirm the endpoint, API/model version, region, identity, and RBAC scope.
398. Validate payload/schema fields and input content type.
399. Check quota, rate limits, network resolution, private connectivity, and service status.
400. Record the corrective action and rerun the acceptance check.

Lab 09 - Extract multimodal information

Source folder: `labs/lab-09-extract-multimodal-information`

Objectives

- Complete the exact Microsoft Learning exercise for the Information extraction domain.
- Capture a working result and explain its product-development implications.

Source

The complete, unchanged Microsoft exercise and assets are in `[official/mslearn-ai-information-extraction/Instructions/Exercises/01-content-understanding.md](../official/mslearn-ai-information-extraction/Instructions/Exercises/01-content-understanding.md)`. The source revision and SHA-256 hashes are recorded in `[OFFICIAL-SOURCE-MANIFEST.json](../OFFICIAL-SOURCE-MANIFEST.json)`. Read the source file for every numbered instruction; this course wrapper does not alter those steps.

Steps

Prepare

Read the source exercise prerequisites and use a trainer-provided Azure subscription or approved lab environment. Do not commit your own `.env` values.

Complete the official exercise

Follow the numbered steps in `[01-content-understanding.md](../official/mslearn-ai-information-extraction/Instructions/Exercises/01-content-understanding.md)` exactly. Use its companion Labfiles or labfiles directory in the same source snapshot.

Capture evidence

Record the deployment or resource names, model and API versions, input, output, latency or quality measure, and one failure or limitation. Remove personal data and live keys.

Validation

The official exercise completes with its expected output; a second learner can reproduce the result from the source instructions and the recorded environment.

Timing

Approximately 35 minutes for the official exercise; trainer discussion and WSQ evidence review are scheduled separately.

Unchanged Microsoft Learning exercise instructions

Exact Markdown source and companion assets:

labs/official/mslearn-ai-information-extraction/Instructions/Exercises/01-content-understanding.md.

Follow the numbered instructions below with the source assets. Microsoft Learning MIT license and pinned revision are in labs/official/ and the source manifest.

In this exercise, you use Azure Content Understanding to extract information from a variety of content types; including an invoice, an image of a slide containing charts, an audio recording of a voice message, and a video recording of a conference call.

This exercise takes approximately 40 minutes.

Create a Microsoft Foundry resource and project

The features we're going to use in this exercise require a Microsoft Foundry resource and project.

401. In a web browser, open the [Microsoft Foundry portal](https://ai.azure.com) at `https://ai.azure.com` and sign in using your Azure credentials. Close any tips or quick start panes that are opened the first time you sign in.
402. Make sure the **New Foundry** toggle is on so that you're using **Foundry (new)**.
403. If you aren't prompted to create a new project automatically, select the project name in the upper-left corner, and then select **Create new project**.
404. Give your project a name and expand **Advanced options** to specify the following settings:
 - Project name: Provide a valid name for your project
 - Foundry resource: Use the default
 - Region: Choose one of the following supported regions:
 - Australia East
 - East US
 - East US 2
 - Japan East
 - North Europe
 - South Central US
 - Southeast Asia

- Sweden Central
- UK South
- West Europe
- West US
- West US 3
- Subscription: Your Azure subscription
- Resource group: Create or select a resource group

> \Azure Content Understanding is available in selected regions. See the [region support documentation](<https://learn.microsoft.com/azure/ai-services/content-understanding/language-region-support>) for the latest availability.

405. Select **Create** and wait for your project to be created.

Download content

The content you're going to analyze is in a .zip archive. Download it and extract it in a local folder.

406. In a new browser tab, download [content.zip](<https://github.com/microsoftlearning/mslearn-ai-information-extraction/raw/main/Labfiles/content/content.zip>) from `'https://github.com/microsoftlearning/mslearn-ai-information-extraction/raw/main/Labfiles/content/content.zip'` and save it in a local folder.
407. Extract the downloaded `*content.zip*` file and view the files it contains. You'll use these files to explore Content Understanding analyzers in this exercise.

> Note: If you're only interested in exploring analysis of a specific modality (documents, images, video, or audio), you can skip to the relevant task below. For the best experience, go through each task to learn how to extract information from different types of content.

Try prebuilt analyzers in Microsoft Foundry

Azure Content Understanding includes prebuilt Read and Layout analyzers that can extract text and structural elements from documents without requiring any custom configuration. These prebuilt analyzers are available directly in the Foundry (new) portal as AI Services models.

Use the Layout analyzer in the playground

408. In the [Microsoft Foundry portal](<https://ai.azure.com>), make sure the **New Foundry** toggle is on.
409. Select **Build** in the upper-right menu, then select **Services** in the left pane.
410. Ensure the **Playgrounds** tab is selected to view the prebuilt models provided by Foundry Tools.
411. Find and select **Content Understanding - Document Layout Analyzer**.

This opens the Layout analyzer playground page, where you can test the layout model on sample data or your own files.

412. In the playground, use the option to upload your own data and upload the **invoice-1234.pdf** file from the folder where you extracted content files. This file contains the following invoice:

![Image of an invoice number 1234.](./media/invoice-1234.png)

413. Run the analyzer and wait for analysis to complete.
414. Review the results. You can view the extracted content either as formatted output or as raw JSON data. Notice that the Layout analyzer extracts text, tables, and structural elements such as paragraphs and sections from the document.

> Note: The prebuilt OCR/Read and Layout analyzers extract content from documents without requiring a generative AI model. OCR/Read extracts text elements (words, paragraphs, formulas, and barcodes), while Layout additionally extracts tables, figures, document structure, hyperlinks, and annotations. These analyzers are useful for general-purpose content extraction, but they don't extract specific custom fields such as invoice amounts or vendor names.

415. Optionally, go back to the **Services** tab and try **Azure Content Understanding - OCR/Read** with the same file to compare the results. Notice that Read extracts text without layout analysis.

Set up Content Understanding Studio for custom analyzers

To extract specific fields from your content (such as invoice amounts, caller names, or meeting participants), you need to build custom analyzers. Custom analyzers are created in Content Understanding Studio, a separate web-based tool for building and testing analyzers with custom schemas.

416. In a new browser tab, open [Content Understanding Studio](https://contentunderstanding.ai.azure.com) at `https://contentunderstanding.ai.azure.com``.
417. If prompted, sign in with the same Azure credentials you used for the Foundry portal.
418. On the **Settings** page (or if redirected to set up your resource), select the **+ Add resource** button.
419. Select the Foundry resource you created earlier, and select **Next** > **Save**.

> Tip: Make sure that the Enable autodeployment for required models if no defaults are available checkbox is selected. This ensures your resource is set up with the required GPT-4.1, GPT-4.1-mini, and text-embedding-3-large models that custom analyzers need.

420. After your resource is connected, you're ready to create custom analyzers. Select **Content Understanding** in the top navigation to go to the home page.

Extract information from invoice documents

You are going to build a custom Azure Content Understanding analyzer that can extract specific fields from invoices. You'll create a project in Content Understanding Studio, define a schema based on a sample invoice, and then build a reusable analyzer.

Create a storage account

Content Understanding Studio requires an Azure Blob Storage account to store the data used for building custom analyzers. You need to create one in the same resource group as your Foundry resource.

421. In a new browser tab, open the [Azure portal](https://portal.azure.com) at `https://portal.azure.com`` and sign in with your Azure credentials.
422. Select **+ Create a resource**, search for `Storage account``, and create a new **Storage account** resource with the following settings:

- Subscription: Your Azure subscription

- Resource group: The same resource group as your Foundry resource
- Storage account name: Enter a globally unique name
- Region: The same region as your Foundry resource
- Preferred storage type: Azure Blob Storage or Azure Data Lake Storage Gen 2
- Performance: Standard
- Redundancy: Locally-redundant storage (LRS)

423. Select **Review + create**, and then **Create**. Wait for deployment to complete.

Define a schema for invoice analysis

424. In Content Understanding Studio, select the **Get started** button in the custom projects section, and select **Create**.

425. Select **Extract content and fields with a custom schema**, then create a project with the following settings:

- Project name: Invoice analysis
- Description: Extract data from an invoice
- Advanced settings
 - Connected resource: Confirm your Foundry resource is selected
 - Connect storage account: Select the storage account you just created
 - Blob container: Create a new container named content-understanding

426. Wait for the project to be created.

> Tip: If an error accessing storage occurs, wait a minute and try again. Permissions for a new resource may take a few minutes to propagate.

427. Upload the **invoice-1234.pdf** file from the folder where you extracted content files.

Content Understanding classifies your data and recommends analyzer templates based on the uploaded content.

428. In the **Choose a template** window, select the **Invoice** template and select **Save**.

The Invoice template includes common fields that are found in invoices. You can use the schema editor to delete any of the suggested fields that you don't need, and add any custom fields that you do.

429. In the list of suggested fields, select **BillingAddress**. This field is not needed for the invoice format you have uploaded, so use the **Delete field** (**🗑**) icon that appears at the end in the selected field row to delete it.

430. In the top bar of the schema tab, select **Suggest**. This will look at the sample invoice and suggest which fields should be a part of your schema. Expand the **Items** field to see which subfields are suggested. Adding those fields will replace your existing schema, so be careful in your projects if you've edited a schema. Select **Save**.

431. Use **+ Add new field** button to add the following field, selecting **Save** (**✓**) for each new field:

| Field name | Field description | Value type | Method |

|--|--|--|

| TotalQuantity | Total number of items on the invoice | String | Auto |

432. Verify that your completed schema looks like this, and select **Save**.

![Screenshot of the invoice analyzer schema in Content Understanding Studio showing fields such as VendorName, InvoiceDate, SubTotal, Items, and TotalQuantity.](./media/invoice-schema.png)

433. Select the **Test** tab, then select **Run analysis** to test your schema. Wait for analysis to complete.

434. Review the analysis results, which should look similar to this:

![Screenshot of invoice analysis test results in Content Understanding Studio showing extracted field values from the sample invoice.](./media/invoice-analysis.png)

435. View the details of the fields that were identified in the **Fields** pane.

Build and test an analyzer for invoices

Now that you have defined a schema to extract fields from invoices, you can build an analyzer to use with similar documents.

436. Select the **Build analyzer** button at the top, and build a new analyzer with the following properties (typed exactly as shown here):

- Name: invoiceanalyzer

- Description: Invoice analyzer

437. When the analyzer has been built, select **Jump to analyzer list** to view all built analyzers, then select the **invoiceanalyzer** link. The fields defined in the analyzer's schema will be displayed.

438. In the **invoiceanalyzer** page, select the **Test** tab.

439. Upload **invoice-1235.pdf** from the folder where you extracted the content files, and run the analysis to extract field data from the invoice.

The invoice being analyzed looks like this:

![Image of an invoice number 1235.](./media/invoice-1235.png)

440. Review the **Fields** pane, and verify that the analyzer extracted the correct fields from the test invoice.

441. Review the **Results** pane to see the JSON response that the analyzer would return to a client application.

442. Close the **invoiceanalyzer** page to return to the analyzer list.

Extract information from a slide image

You are going to build a custom Azure Content Understanding analyzer that can extract information from a slide containing charts.

Define a schema for image analysis

443. In **Project list** tab, select **Create** and select **Extract content and fields with a custom schema**, then create a project with the following settings:

- Project name: Slide analysis
- Description: Extract data from an image of a slide
- Advanced settings: Verify the settings are the same as the last project

444. Wait for the project to be created.

445. Upload the **slide-1.jpg** file from the folder where you extracted content files. Then select the **Image analysis** template and select **Save**.

The Image analysis template doesn't include any predefined fields. You must define fields to describe the information you want to extract.

446. Use the **+ Add new field** button to add the following fields, selecting **Save changes** (**✓**) for each new field:

Field name	Field description	Value type	Method
Title	Slide title	String	Generate
Summary	Summary of the slide	String	Generate
Charts	Number of charts on the slide	Integer	Generate

447. Use **+ Add new field** button to add a new field named `QuarterlyRevenue` with the description `Revenue per quarter` with the value type **List of objects**. Then, select the table icon next to the value type dropdown. In the new page for the table subfields that opens, add the following subfields:

Field name	Field description	Value type	Method
Quarter	Which quarter?	String	Generate
Revenue	Revenue for the quarter	Number	Generate

448. Select **Back** to return to the top level of your schema, and use **+ Add new field** button to add a new field named `ProductCategories` with the description `Product categories` with the value type **List of objects**. Then, select the table icon next to the value type to open a new page for the table subfields, add the following subfields:

Field name	Field description	Value type	Method
ProductCategory	Product category name	String	Generate
RevenuePercentage	Percentage of revenue	Number	Generate

449. Select **Back** to return to the top level of your schema, and verify that it looks like this. Then select **Save**.

!Screenshot of the image analyzer schema in Content Understanding Studio showing fields for Title, Summary, Charts, QuarterlyRevenue, and ProductCategories.](./media/slide-schema.png)

450. Select the **Test** tab, then **Run analysis** and wait for analysis to complete.
451. Review the analysis results, which should look similar to this:

![Screenshot of image analysis test results in Content Understanding Studio showing extracted fields from the slide including revenue data and product categories.](./media/slide-analysis.png)

452. View the details of the fields that were identified in the **Fields** pane, expanding the **QuarterlyRevenue** and **ProductCategories** fields to see the subfield values.

Build and test an analyzer

Now that you have defined a schema to extract fields from slides, you can build an analyzer to use with similar slide images.

453. Select the **Build analyzer** button at the top, and build a new analyzer with the following properties (typed exactly as shown here):

- Name: slideanalyzer
- Description: Slide image analyzer

454. When the analyzer has been built, select **Jump to analyzer list**, then select the **slideanalyzer** link. The fields defined in the analyzer's schema will be displayed.

455. In the **slideanalyzer** page, select the **Test** tab.

456. Use the **+ Upload test files** button to upload **slide-2.jpg** from the folder where you extracted the content files, and run the analysis to extract field data from the image.

457. Review the **Fields** pane, and verify that the analyzer extracted the correct fields from the slide image.

> Note: Slide 2 doesn't include a breakdown by product category, so the product category revenue data is not found.

458. Review the **Results** pane to see the JSON response that the analyzer would return to a client application.
459. Close the **slideanalyzer** page.

Extract information from a voicemail audio recording

You are going to build a custom Azure Content Understanding analyzer that can extract information from an audio recording of a voicemail message.

Define a schema for audio analysis

460. In **Project list** tab, select **Create** and select **Extract content and fields with a custom schema**, then create a project with the following settings:

- Project name: Voicemail analysis
- Description: Extract data from a voicemail recording
- Advanced settings: Verify the settings are the same as the last project

461. Wait for the project to be created.

462. Upload the **call-1.mp3** file from the folder where you extracted content files. Then select the **Audio analysis** template and select **Save**.

463. In the **Content** pane on the right, select **Get transcription preview** to see a transcription of the recorded message.

The Audio analysis template doesn't include any predefined fields. You must define fields to describe the information you want to extract.

464. Use **++ Add new field++** button to add the following fields, selecting ****Save**** (****✓****) for each new field:

Field name	Field description	Value type	Method
Caller	Person who left the message	String	Generate
Summary	Summary of the message	String	Generate
Actions	Requested actions	String	Generate
CallbackNumber	Telephone number to return the call	String	Generate
AlternativeContacts	Alternative contact details	List of Strings	Generate

465. Select ****Run analysis**** and wait for analysis to complete.

Audio analysis can take some time. While you're waiting, you can play the audio file below:

```
<video controls src="/media/call-1.mp4" title="Call 1" width="300">
  <track src="/media/call-1.vtt" kind="captions" srclang="en" label="English">
</video>
```

Note: This audio was generated using AI.

466. Review the analysis results and view the details of the fields that were identified in the ****Fields**** pane, expanding the ****AlternativeContacts**** field to see the listed values.

Build and test an analyzer

Now that you have defined a schema to extract fields from voice messages, you can build an analyzer to use with similar audio recordings.

467. Select the ****Build analyzer**** button at the top, and build a new analyzer with the following properties (typed exactly as shown here):

- Name: voicemailanalyzer
- Description: Voicemail audio analyzer

468. When the analyzer has been built, select ****Jump to analyzer list****, then select the ****voicemailanalyzer**** link. The fields defined in the analyzer's schema will be displayed.

469. In the ****voicemailanalyzer**** page, select the ****Test**** tab.

470. Use the **++ Upload test files++** button to upload ****call-2.mp3**** from the folder where you extracted the content files, and run the analysis to extract field data from the audio file.

Audio analysis can take some time. While you're waiting, you can play the audio file below:

```
<video controls src="/media/call-2.mp4" title="Call 2" width="300">
  <track src="/media/call-2.vtt" kind="captions" srclang="en" label="English">
```

</video>

Note: This audio was generated using AI.

471. Review the **Fields** pane, and verify that the analyzer extracted the correct fields from the voice message.
472. Review the **Results** pane to see the JSON response that the analyzer would return to a client application.
473. Close the **voicemail-analyzer** page.

Extract information from a video conference recording

You are going to build a custom Azure Content Understanding analyzer that can extract information from a video recording of a conference call.

Define a schema for video analysis

474. In Content Understanding Studio, select **Create project** on the home page (or use the navigation to return to the home page first).
475. Select **Extract content and fields with a custom schema**, then create a project with the following settings:
 - Project name: Conference call video analysis
 - Description: Extract data from a video conference recording
476. Wait for the project to be created.
477. Upload the **meeting-1.mp4** file from the folder where you extracted content files. Then select the **Video analysis** template and select **Create**.
478. In the **Content** pane on the right, select **Get transcription preview** to see a transcription of the recorded meeting.

The Video analysis template extracts data for each segment. It doesn't include any predefined fields. You must define fields to describe the information you want to extract.

479. Use **+ Add new field** button to add the following fields, selecting **Save** (**✓**) for each new field:

Field name	Field description	Value type	Method
------------	-------------------	------------	--------

--	--	--	--

Summary	Summary of the discussion	String	Generate
---------	---------------------------	--------	----------

Participants	Count of meeting participants	Integer	Generate
--------------	-------------------------------	---------	----------

ParticipantNames	Names of meeting participants	List of Strings	Generate
------------------	-------------------------------	-----------------	----------

SharedSlides	Descriptions of any PowerPoint slides presented	List of Strings	Generate
--------------	---	-----------------	----------

AssignedActions	Tasks assigned to participants	List of Objects	Generate
-----------------	--------------------------------	-----------------	----------

480. When you enter the **AssignedActions** field, in the table of subfields, create the following subfields:

Field name	Field description	Value type	Method
------------	-------------------	------------	--------

--	--	--	--

| Task | Description of the task | String | Generate |

| AssignedTo | Who the task is assigned to | String | Generate |

481. Select **Back** to return to the top level of your schema, and verify that it looks like this. Then select **Save**.

482. Select **Run analysis** and wait for analysis to complete.

Video analysis can take some time. While you're waiting, you can view the video below:

```
<video controls src="/media/meeting-1.mp4" title="Meeting 1" width="480">  
  <track src="/media/meeting-1.vtt" kind="captions" srclang="en" label="English">  
</video>
```

Note: This video was generated using AI.

483. When analysis is complete, review the results.

484. In the **Fields** pane, view the extracted data.

Build and test an analyzer

Now that you have defined a schema to extract fields from conference call recordings, you can build an analyzer to use with similar videos.

485. Select the **Build analyzer** button at the top, and build a new analyzer with the following properties (typed exactly as shown here):

- Name: meetinganalyzer

- Description: Meeting video analyzer

486. Wait for the new analyzer to be ready (use the **Refresh** button to check).

487. When the analyzer has been built, select **Jump to analyzer list**, then select the **meetinganalyzer** link. The fields defined in the analyzer's schema will be displayed.

488. In the **meetinganalyzer** page, select the **Test** tab.

489. Use the **+ Upload test files** button to upload **meeting-2.mp4** from the folder where you extracted the content files, and run the analysis to extract field data from the video file.

Video analysis can take some time. While you're waiting, you can view the video below:

```
<video controls src="/media/meeting-2.mp4" title="Meeting 2" width="480">  
  <track src="/media/meeting-2.vtt" kind="captions" srclang="en" label="English">  
</video>
```

Note: This video was generated using AI.

490. Review the **Fields** pane, and view the fields that the analyzer extracted for each shot in the conference call video.

491. Review the **Results** pane to see the JSON response that the analyzer would return to a client application.

492. Close the **meetinganalyzer** page.

Clean up

If you've finished working with the Content Understanding service, you should delete the resources you have created in this exercise to avoid incurring unnecessary Azure costs.

493. In the [Azure portal](https://portal.azure.com), delete the resource group you created for this exercise.

Acceptance Evidence

Complete the pinned Microsoft Learning exercise `official/mslearn-ai-information-extraction/Instructions/Exercises/01-content-understanding.md`; capture its expected output, relevant version/endpoint, one measured result, and an explained failure or limitation.

Troubleshooting Checklist

494. Capture the full error and request/operation ID before retrying.
495. Confirm the endpoint, API/model version, region, identity, and RBAC scope.
496. Validate payload/schema fields and input content type.
497. Check quota, rate limits, network resolution, private connectivity, and service status.
498. Record the corrective action and rerun the acceptance check.

Lab 10 - Build a knowledge mining solution

Source folder: `labs/lab-10-build-a-knowledge-mining-solution`

Objectives

- Complete the exact Microsoft Learning exercise for the Information extraction domain.
- Capture a working result and explain its product-development implications.

Source

The complete, unchanged Microsoft exercise and assets are in `[official/mslearn-ai-information-extraction/Instructions/Exercises/04-knowledge-mining.md](../official/mslearn-ai-information-extraction/Instructions/Exercises/04-knowledge-mining.md)`. The source revision and SHA-256 hashes are recorded in `[OFFICIAL-SOURCE-MANIFEST.json](../OFFICIAL-SOURCE-MANIFEST.json)`. Read the source file for every numbered instruction; this course wrapper does not alter those steps.

Steps

Prepare

Read the source exercise prerequisites and use a trainer-provided Azure subscription or approved lab environment. Do not commit your own `.env` values.

Complete the official exercise

Follow the numbered steps in `[04-knowledge-mining.md](../official/mslearn-ai-information-extraction/Instructions/Exercises/04-knowledge-mining.md)` exactly. Use its companion `Labfiles` or `labfiles` directory in the same source snapshot.

Capture evidence

Record the deployment or resource names, model and API versions, input, output, latency or quality measure, and one failure or limitation. Remove personal data and live keys.

Validation

The official exercise completes with its expected output; a second learner can reproduce the result from the source instructions and the recorded environment.

Timing

Approximately 45 minutes for the official exercise; trainer discussion and WSQ evidence review are scheduled separately.

Unchanged Microsoft Learning exercise instructions

Exact Markdown source and companion assets:

labs/official/mslearn-ai-information-extraction/Instructions/Exercises/04-knowledge-mining.md.

Follow the numbered instructions below with the source assets. Microsoft Learning MIT license and pinned revision are in labs/official/ and the source manifest.

In this exercise, you use Azure AI Search to create a knowledge mining solution that indexes a set of travel brochure documents. The indexing process uses AI skills to extract key information from the documents, and you'll create a Python client application to search the index.

This exercise takes approximately 40 minutes.

Create Azure resources

The solution requires multiple resources in your Azure subscription, all created in the same region.

Create an Azure AI Search resource

499. In a web browser, open the [Azure portal](https://portal.azure.com) at `https://portal.azure.com` and sign in with your Azure credentials.

500. Select the **Create a resource** button, search for 'Azure AI Search', and create an **Azure AI Search** resource with the following settings:

- Subscription: Your Azure subscription
- Resource group: Create or select a resource group
- Service name: A valid name for your search resource
- Location: Any available location
- Pricing tier: Free

501. Wait for deployment to complete, and then go to the deployed resource.

502. Review the **Overview** page. Here you can use a visual interface to create, test, manage, and monitor the various components of a search solution.

Create a storage account

503. Return to the Azure portal home page and create a **Storage account** resource with the following settings:

- Subscription: Your Azure subscription
- Resource group: The same resource group as your Azure AI Search resource
- Storage account name: A valid name for your storage resource
- Region: The same region as your Azure AI Search resource

- Primary service: Azure Blob Storage or Azure Data Lake Storage Gen 2
- Performance: Standard
- Redundancy: Locally-redundant storage (LRS)

504. Wait for deployment to complete, and then go to the deployed resource.

Upload documents to Azure Storage

Your knowledge mining solution will extract information from travel brochure documents stored in Azure Blob Storage.

505. In a new browser tab, download [documents.zip](https://github.com/microsoftlearning/mslearn-ai-information-extraction/raw/main/Labfiles/knowledge/documents.zip) from `https://github.com/microsoftlearning/mslearn-ai-information-extraction/raw/main/Labfiles/knowledge/documents.zip` and save it to a local folder.
506. Extract the downloaded *documents.zip* file and view the travel brochure files it contains.
507. In the Azure portal, navigate to your storage account and select **Storage browser** in the navigation pane.
508. In the storage browser, select **Blob containers**.
509. In the toolbar, select **+ Container** and create a new container with the following settings:
- Name: documents
 - Anonymous access level: Private (no anonymous access)
510. Select the **documents** container, and use the **Upload** toolbar button to upload the .pdf files you extracted from **documents.zip**.

Create and run an indexer

Now that you have the documents in place, you can create an indexer to use AI skills to extract information from them.

511. In the Azure portal, browse to your Azure AI Search resource. On its **Overview** page, select **Import data**.
512. On the **Connect to your data** page, in the **Data Source** list, select **Azure Blob Storage**.
513. Select **keyword search**. Then complete the data store details with the following values:
514. On **Connect to your data** form set the following:
- Storage account: Your recently created storage account
 - Blob container: Select the documents container.
 - Leave the remaining options as their default values, and then select Next.
515. On **Apply AI enrichments** set the following:
- Select Extract phrases.
 - Select Extract entities, select the settings icon, ensure only Persons and Locations are selected, and then select Save.
 - Select Extract text from images, select the settings icon, ensure Generate tags and Categorize content are selected, and then select Save.

- If it isn't already selected, choose the free Foundry Tools resource option, and then select Next.

> Note: The free Azure AI Services enrichment for Azure AI Search can be used to index a maximum of 20 documents. In a production solution, you should create and attach an Azure AI Services resource.

516. On **Preview mappings** set the following configuration:

- The fields are already mapped based on the options you selected in the previous step.
- Review the following fields and ensure that they're configured as shown in the following table. To update a field, select it and then select Configure field. Leave all other fields with their default settings.

Target index field name	Retrievable	Filterable	Sortable	Facetable	Searchable
-----	-----	-----	-----	-----	-----
metadata_storage_size	✔	✔	✔		
metadata_storage_last_modified	✔	✔	✔		
title	✔	✔	✔	✔	
locations	✔	✔	✔		
persons	✔	✔	✔		
keyPhrases	✔	✔	✔		

- Double-check your selections carefully.

- Select Next.

517. On **Advanced settings** set the following:

- Ensure Enable semantic ranker is selected.
- If it isn't already selected, set Schedule to Once.
- Select Next.

518. On **Review and create** set **Objects name prefix** to `margies-index` and then select **Create**.

519. You may close the success notification.

520. In the navigation pane on the left, under **Search management**, view the **Indexers** page. The **margies-index-indexer** should appear. Wait a few minutes, and click **↻ Refresh** until the **Status** indicates **Success**.

Search the index

Now that you have an index, you can search it.

521. Return to the **Overview** page for your Azure AI Search resource, and on the toolbar, select **Search explorer**.

522. In Search explorer, in the **Query string** box, enter `\*` (a single asterisk) and then select **Search**.

This query retrieves all documents in the index in JSON format. Examine the results and note the fields for each document, which include document content, metadata, and enriched data extracted by the cognitive skills.

523. In the **View** menu, select **JSON view** and note that the JSON request for the search is shown:

```
json
{
  "search": "",
  "count": true
}
```

524. The results include a **@odata.count** field at the top of the results that indicates the number of documents returned by the search.

525. Modify the JSON request to include a **select** parameter:

```
json
{
  "search": "",
  "count": true,
  "select": "title,locations"
}
```

This time the results include only the file name and any locations mentioned in the document content. The file name is in the title field. The locations field was generated by an AI skill.

526. Try the following query string:

```
json
{
  "search": "New York",
  "count": true,
  "select": "title,keyPhrases"
}
```

This search finds documents that mention "New York" in any searchable field, and returns the file name and key phrases.

527. Try one more query:

```
json
{
  "search": "New York",
  "count": true,
  "select": "title,keyPhrases",
  "filter": "metadata_storage_size lt 380000"
}
```

This returns documents mentioning "New York" that are smaller than 380,000 bytes.

Create a search client application

Now that you have a useful index, you can query it from a Python client application using the Azure AI Search SDK.

Get the endpoint and keys for your search resource

528. In the Azure portal, return to the **Overview** page for your Azure AI Search resource. Note the **Url** value (e.g., `https://your_resource_name.search.windows.net`). This is the endpoint for your search resource.
529. In the navigation pane on the left, expand **Settings** and view the **Keys** page. Note the **query** key — you'll need this for your client application.

> Note: Azure AI Search creates one default query key for the service. In the Azure portal, this default query key can appear with a blank name. This is expected behavior.

Prepare to use the Azure AI Search SDK

530. Start **Visual Studio Code**.
531. Open the Command Palette (press **Ctrl+Shift+P**), type **Git: Clone**, and select it.
532. In the URL bar, paste the following repository URL and press **Enter**:

```
https://github.com/microsoftlearning/mslearn-ai-information-extraction
```

533. Choose a local folder to clone into, and then when prompted, select **Open** to open the cloned repository in VS Code.
534. Open a new terminal and navigate to the Python code folder:

```
cd Labfiles/04-knowledge-mining
```

535. Install the required packages:

```
python -m venv labenv
```

```
labenv\Scripts\activate
```

```
pip install -r requirements.txt
```

> Note: The requirements.txt installs the [azure-search-documents](<https://learn.microsoft.com/python/api/overview/azure/search-documents-readme?view=azure-python>) Python SDK package and its dependencies.

536. In the VS Code Explorer pane, open the **.env** file in **Labfiles/04-knowledge-mining**.

537. Replace the following placeholder values:

- your_search_endpoint: The endpoint for your Azure AI Search resource
- your_query_key: The query key for your Azure AI Search resource
- your_index_name: The name of your index (should be margies-index)

538. Save the file (**CTRL+S**).

539. In VS Code, open the **search-app.py** file.

540. Review the code, which:

- Retrieves the configuration settings from the .env file.
- Creates a SearchClient with the endpoint, key, and index name.
- Prompts the user for a search query in a loop (until they type "quit").
- Searches the index using the query, returning the following fields ordered by title:
 - title
 - locations
 - persons
 - keyPhrases
- Parses the search results that are returned to display the fields returned for each document in the result set.

541. In the VS Code terminal, run the application:

```
python search-app.py
```

542. When prompted, enter a query such as `London` and view the results.

543. Try another query, such as `flights`.

544. When you're finished testing, enter `quit` to close the app.

Note about knowledge store

Knowledge store steps are excluded from this version of the exercise.

The current Import data keyword search flow in the Azure portal doesn't create a knowledge store for this scenario, and the multimodal alternative hasn't been adopted for this exercise.

Clean up

If you've finished working with Azure AI Search, you should delete the resources you created in this exercise to avoid incurring unnecessary Azure costs.

545. In the [Azure portal](https://portal.azure.com), delete the resource group you created for this exercise.

More information

To learn more about Azure AI Search, see the [Azure AI Search documentation](https://docs.microsoft.com/azure/search/search-what-is-azure-search).

Acceptance Evidence

Complete the pinned MicrosoftLearning exercise [official/mslearn-ai-information-extraction/Instructions/Exercises/04-knowledge-mining.md](https://www.microsoft.com/learning/exercise/official/mslearn-ai-information-extraction/Instructions/Exercises/04-knowledge-mining.md); capture its expected output, relevant version/endpoint, one measured result, and an explained failure or limitation.

Troubleshooting Checklist

- 546. Capture the full error and request/operation ID before retrying.
- 547. Confirm the endpoint, API/model version, region, identity, and RBAC scope.
- 548. Validate payload/schema fields and input content type.
- 549. Check quota, rate limits, network resolution, private connectivity, and service status.
- 550. Record the corrective action and rerun the acceptance check.

Quick Command Reference

Command / item	Purpose
az login --use-device-code	Authenticate Azure CLI without embedding credentials
az account show --output table	Confirm active tenant/subscription
python -m venv .venv	Create an isolated Python environment
az monitor metrics list ...	Retrieve a metric for operational evidence
git status --short	Confirm no secrets or generated private assessment files are staged

Assessment Flow and Support

- 551. Complete TRAQOM digital attendance.
- 552. Complete Assessment digital attendance.
- 553. Sit the 60-minute WA-SAQ, then the 60-minute PP.
- 554. Submit the two candidate papers on the LMS.
- 555. Sign the Assessment Summary Record.

Support: enquiry@tertiaryinfotech.com · +65 6100 0613 · <https://lms-tms.tertiaryinfotech.com/>